

Chapter 1

Interpolation in Knowledge Representation

Jean Christoph Jung ✉ 

Department of Computer Science, TU Dortmund University, Germany

Patrick Koopmann ✉ 

Vrije Universiteit Amsterdam, Netherlands

Matthias Knorr ✉ 

Universidade Nova de Lisboa, Portugal

Abstract

Craig interpolation and uniform interpolation have many applications in knowledge representation, including explainability, forgetting, modularization and reuse, and even learning. At the same time, many relevant knowledge representation formalisms do in general not have Craig or uniform interpolation, and computing interpolants in practice is challenging. We have a closer look at two prominent knowledge representation formalisms, description logics and logic programming, and discuss theoretical results and practical methods for computing interpolants.

Contents

1	Introduction	2
2	Foundations of Description Logics	6
3	Uniform Interpolation for Description Logic Ontologies	8
3.1	Existence and Size	10
3.2	Computing Uniform Interpolants in Practice	14
3.3	Related Notions and Applications	18
4	Craig Interpolation for Description Logic Concepts	20
4.1	Craig Interpolation	20
4.2	Beth Definability	26
4.3	Applications	29
4.4	Computing Interpolants	30
5	Interpolation in Logic Programming	33
5.1	Logic Programs and Answer Sets	33
5.2	Craig Interpolants	35

 © 2025 Copyright for this paper by its authors
DRAFT

Final version to appear in Balder ten Cate, Jean Christoph Jung, Patrick Koopmann, Christoph Wernhard and Frank Wolter, editors. *Theory and Applications of Craig Interpolation*, chapter XXXCHAPTER, pages XXXPFROM–XXXPTO. Ubiquity Press, 2026. XXXDOI.

2 Interpolation in Knowledge Representation

5.3 Uniform Interpolation and Forgetting	36
6 Conclusion	39
Acknowledgments	40
References	40

1 Introduction

The field of Knowledge Representation and Reasoning (KR) deals with the explicit representation and manipulation of knowledge in a format that is both *machine-processable* and *human-readable*, and plays a central role in symbolic AI [123, 21]. AI systems using KR are referred to as *knowledge-based systems*. Depending on the application, they use different *knowledge representation formalisms* satisfying the mentioned two conditions. Since interpolation is an inherently logical notion, in this chapter, we will concentrate on logic-based formalisms, and neglect, e.g., graph-based KR formalisms such as argumentation frameworks [14] or probabilistic graphical models [69]. Knowledge-based systems then implement inference procedures tailored to the underlying logic and use them to make decisions.

Let us illustrate the idea underlying KR with a simple example, formulated in terms of first-order logic. The following statements could be part of a formalization of our knowledge about cars that (1) every car has a prime mover, (2) the prime mover can be a diesel engine, a gas engine or an electric motor, and (3) every car with an electric motor is an electric car:

$$\forall x (\text{Car}(x) \rightarrow \exists y (\text{hasPart}(x, y) \wedge \text{PrimeMover}(y))) \quad (1)$$

$$\forall x (\text{PrimeMover}(x) \rightarrow (\text{DieselEngine}(x) \vee \text{GasEngine}(x) \vee \text{ElectricMotor}(x))) \quad (2)$$

$$\forall x (\text{Car}(x) \wedge \exists y (\text{hasPart}(x, y) \wedge \text{ElectricMotor}(y)) \rightarrow \text{ElectricCar}(x)) \quad (3)$$

If we now find out that a car c has an electric motor, we can feed this knowledge and additional facts

$$\text{Car}(c), \quad \text{hasPart}(c, e), \quad \text{ElectricMotor}(e)$$

into the inference procedure, which allows us to derive that c is an electrical car.

Interpolation plays a central role in many applications of KR. To ease the discussion, we recall the definition of Craig and uniform interpolants, formulated here for first-order logic (FO) with entailment relation $\models$. Here, and in the remainder of the introduction, a *signature* is a set of non-logical symbols.

► **Definition 1.** A Craig interpolant for FO-formulae $\phi, \psi \in \text{is}$ a formula $\chi \in \text{FO}$ such that

- χ uses only non-logical symbols occurring in both ϕ and ψ , and
- $\phi \models \chi, \chi \models \psi$.

Let Σ be signature. A uniform Σ -interpolant for ϕ is an FO-formula ϕ_Σ such that:

- $\phi \models \phi_\Sigma$;
- ϕ_Σ uses only non-logical symbols from Σ ;
- for any FO-formula ψ , if $\phi \models \psi$ and ψ uses only non-logical symbols from Σ , then $\phi_\Sigma \models \psi$.

In applications, the formula ϕ from Definition 1 will typically be the modeled knowledge, which is why we will denote it with $\mathcal{K}$, for *knowledge base*. Although knowledge bases come in different forms for different KR formalisms, we refrain from specifying the exact formalism in the applications below, as we only want to convey the general ideas. We start with applications of uniform interpolants.

Forgetting and Information Hiding. The most classical connection between interpolation and KR might be *forgetting*, originally introduced for first-order logic in a seminal paper by Lin and Reiter [87]. The idea of forgetting is that there are non-logical symbols that are considered “obsolete” and should be “forgotten” from the current knowledge in a way that preserves as much information as possible. There are different definitions of forgetting that can be found in the KR literature, see [30] for a survey, but the most common one is equivalent to uniform interpolation: forgetting a symbol X from a knowledge base $\mathcal{K}$ corresponds to computing a uniform interpolant of $\mathcal{K}$ for the signature $\text{sig}(\mathcal{K}) \setminus \{X\}$. Forgetting is also useful for applications that require some form of *information hiding*, such as knowledge publication and exchange. For instance, if we were to publish $\mathcal{K}$ without disclosing anything about a private signature Σ , then a uniform $(\text{sig}(\mathcal{K}) \setminus \Sigma)$ -interpolant of $\mathcal{K}$ would be a good candidate, as it contains the “maximal” information contained in $\mathcal{K}$ about the remaining signature [54].

Abduction. Abduction is a classical problem in KR that can be formalized as follows: given a knowledge base $\mathcal{K}$ and an *observation* ψ such that $\mathcal{K} \not\models \psi$, we want to find a *hypothesis* $\mathcal{H}$ satisfying $\mathcal{K} \wedge \mathcal{H} \models \psi$. The hypothesis $\mathcal{H}$ can be seen as a possible explanation for some unexpected phenomenon ϕ , a diagnosis, or an indication of how to complete an incomplete knowledge base. To avoid trivial solutions, one typically formulates additional requirements on the hypothesis such as certain minimality conditions or a signature Σ from which the hypothesis ought to be constructed. It is immediate from the definitions that the negation of a Σ -uniform interpolant of $\mathcal{K} \wedge \neg\psi$ is a *logically weakest* hypothesis, that is, it is entailed by all other alternative hypotheses over signature Σ . This connection has been exploited in several abduction systems [40, 28, 78].

Modularisation and Reuse. Knowledge bases often contain tens or even hundreds of thousands of statements (e.g. [29, 74]) which can make it challenging to work with them. At the same time, it could be that for a particular application, only a fragment of the knowledge base is actually relevant. Uniform interpolants computed for a restricted, user-given signature can provide a more focussed view on the knowledge that is relevant to the application at hand, and can be used as a replacement of the original knowledge base. But uniform interpolants can also be used to determine whether a subset of the knowledge base preserves all entailments over a relevant signature, which can be used to *modularize* the knowledge base into different components based on a signature.

4 Interpolation in Knowledge Representation

Analysis and Summarization. Uniform interpolants for small signatures Σ can make hidden relations between the symbols in Σ explicit, and thus help knowledge engineers in analyzing how different symbols in the knowledge base relate to each other. We can also use uniform interpolation to summarize large knowledge bases, for instance by choosing for Σ a set of symbols that is central to the knowledge base (e.g. most frequently used). The interpolant then gives a high-level perspective on the content of the knowledge base, which can be seen as a summary.

Craig interpolants have a range of applications for KR as well.

Explaining Entailments. One of the main benefits of KR is that the use of explicit representations of knowledge with a well-defined semantics leads to a higher transparency and understandability of the knowledge-based system. Still, for large knowledge bases and expressive KR formalisms, inferences may not be straightforward, which has motivated many researchers to investigate methods for explaining logical inferences for KR systems [100, 115, 32, 122, 3]. Craig interpolants are one way of explaining logical inferences: in particular, to explain an entailment $\mathcal{K} \models \psi$, we can provide a Craig interpolant for $\mathcal{K}, \psi$, which highlights the reason for the entailment in the common signature of $\mathcal{K}$ and ψ . Notice that the existence of a Craig interpolant for $\mathcal{K}, \psi$ is a weaker requirement than the existence of a uniform interpolant for $\mathcal{K}$, and indeed Craig interpolants can be computed in more cases.

Explicit Definitions. It is well-known that Craig interpolation is closely related to the notion of *Beth definability*. In the context of KR, this connection is particularly relevant for the problem of finding *explicit definitions* of predicates which are *implicitly defined*. An explicit definition of a unary predicate A in knowledge base $\mathcal{K}$ is a formula $\chi(x)$ not mentioning A with $\mathcal{K} \models \forall x (A(x) \leftrightarrow \chi(x))$. That is, χ provides a direct description of the *meaning* of A , and we might add this definition to $\mathcal{K}$. A special kind of explicit definitions are *referring expressions*. Referring expressions are a concept originating in linguistics and are phrases that refer to specific objects by providing a unique description for them, as in “the current president of the USA” or “the capital of France”. In a knowledge base $\mathcal{K}$, a referring expression for a constant c can be viewed as an explicit definition of $x = c$ over $\mathcal{K}$. The use of referring expressions in KR and data management has been advocated for instance in [7, 80, 19, 9].

Separating Examples and Learning Logical Formulas. Consider the following separation problem. Suppose we are given two sets P and N of constants occurring in a knowledge base $\mathcal{K}$ that respectively represent *positive* and *negative* examples. We are then looking for a logical formula $\chi(x)$ that *separates* P from N over $\mathcal{K}$ in the sense that $\mathcal{K} \models \chi(a)$ for all $a \in P$ and $\mathcal{K} \models \neg\chi(b)$ for all $b \in N$. Such separation problems have been investigated thoroughly in KR in the context of learning logical formulas [66, 36]. Recently, it has been observed that in relevant cases, the problem of finding a separating formula is interreducible with the problem of computing Craig interpolants [8].

Motivated by these applications, there has been an enormous interest in studying interpolation in different formalisms. The baseline logical formalisms used in KR are certainly

the classical *propositional logic* and *first-order logic*, which have the benefit of a clean and well-understood semantics. However, for many realistic applications, neither of them is well-suited, since they are either limited in their expressivity, or do not admit time-efficient reasoning. As a consequence, a wealth of other formalisms have been introduced that are tailored towards specific applications. Examples include description logics, modal and temporal logics, logic programs, default logic, existential rules, planning languages, and many more [123]. Interpolation is useful in many of these formalisms, but it has not been equally investigated in all of them. Interpolation in propositional logic is covered in Chapter 1, interpolation in modal logic is covered in Chapter 4, and interpolation in first-order logic is covered in Chapter 2 and Chapter 12. The majority of research on interpolation in the remaining formalisms has been conducted in description logics and logic programming, which is why we concentrate on these two formalisms in this chapter:

- *Description Logics (DLs)* are a family of KR languages commonly used to formalize ontologies [11]. Here, an *ontology* is a formal specification of the concepts and their relations in some domain of interest such as biology or medicine. An example ontology (in first-order logic) is provided in Equations (1)–(3) above. Ontologies are important in information science, since they can be used by different parties to share knowledge about that domain, which has been exploited, for example, in biology, medicine, and artificial intelligence [74, 29, 41, 27]. DLs are also highly relevant in the *Semantic Web* [61]. Indeed, they form the logical basis of the W3C standard *web ontology language* OWL [62]. Typically, DLs are fragments of first-order logic with decidable inference problems, which makes them suitable for the mentioned applications.
- *Logic programming* is concerned with the use of *logical rules* to represent knowledge. One of the main features is that inference is *nonmonotonic*, that is, extending the knowledge base can invalidate previously made inferences. This is in contrast to *monotonic* logics such as first-order logic (and hence DLs as well). While logic programming has been studied since the 1960s, it is still relevant today, in particular in the form of *answer set programming* (ASP), which is a declarative approach to modeling and solving combinatorial problems [43]. It plays a central role in many applications, for instance for solving configuration problems.

It is particularly remarkable how much has been done on the topic of interpolation in DLs. We conjecture that this is due to the fact that the main applications of DLs are ontologies describing the conceptualization of a domain of discourse, a task that is ultimately linked with the signature, which is also central to interpolation. In the chapter, this will be reflected by the fact that we will mainly focus on interpolation for DLs, and only briefly discuss interpolation for logic programming. A peculiarity in the literature on interpolation in DLs, which has to do with the applications, is that uniform interpolation has been investigated mostly for the case where the knowledge base is an ontology, and Craig interpolation for DLs has been mostly investigated for concept descriptions, in which case the ontology is treated as background theory.

6 Interpolation in Knowledge Representation

Looking at the applications, it appears that the central problem to be investigated is the *computation* of Craig and uniform interpolants. Unfortunately, their existence is by no means guaranteed in the relevant logics, so another focus of the chapter will be the corresponding *existence* problems, and the practically relevant question of what to do if no (Craig or uniform) interpolant exists. Given the technical similarity of description logics and modal logics, we will discuss the concrete relation (in terms of interpolation) when appropriate.

The chapter is structured as follows. In Section 2, we recall the necessary foundations of description logics. In Section 3, we discuss uniform interpolation for the case where the knowledge base is a description logic ontology, looking at both theoretical results and practical methods for computing interpolants. In Section 4, we discuss results on interpolation and Beth definability for DL concept descriptions from a more theoretical perspective. In Section 5, we discuss the role of Craig interpolants in logic programming, as well as uniform interpolation and forgetting. Finally, in Section 6, we conclude the chapter and provide an outlook for future directions.

2 Foundations of Description Logics

We first introduce the syntax and semantics of the basic description logic $\mathcal{ALC}$, discuss the extensions/restrictions relevant for the chapter, and introduce some model theory. We refer the reader to [11] for a comprehensive introduction to description logics. Let $\mathbf{N}_C$, $\mathbf{N}_R$, and $\mathbf{N}_I$ be mutually disjoint and countably infinite sets of *concept*, *role*, and *individual names*. An $\mathcal{ALC}$ *concept* is defined according to the syntax rule

$$C, D ::= \top \mid A \mid \neg C \mid C \sqcap D \mid C \sqcup D \mid \forall r.C \mid \exists r.C$$

where A ranges over concept names and r over role names. We use the abbreviations $C \rightarrow D$ and $C \leftrightarrow D$ for $\neg C \sqcup D$, and $(C \rightarrow D) \sqcap (D \rightarrow C)$, respectively. An $\mathcal{ALC}$ *concept inclusion* ($\mathcal{ALC}$ *CI*) takes the form $C \sqsubseteq D$ for $\mathcal{ALC}$ concepts C and D . An $\mathcal{ALC}$ *ontology* is a finite set of $\mathcal{ALC}$ CIs. We drop the reference to $\mathcal{ALC}$ if no confusion can arise.

The semantics is defined in terms of *interpretations* $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$, where $\Delta^{\mathcal{I}}$ is a non-empty set, called *domain* of $\mathcal{I}$, and $\cdot^{\mathcal{I}}$ is a function mapping every $A \in \mathbf{N}_C$ to a subset $A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$, every $r \in \mathbf{N}_R$ to a subset $r^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$, and every $a \in \mathbf{N}_I$ to an element in $\Delta^{\mathcal{I}}$. Moreover, the *extension* $C^{\mathcal{I}}$ of a concept C in $\mathcal{I}$ is defined as follows, where r ranges over role names:

$$\begin{aligned} \top^{\mathcal{I}} &= \Delta^{\mathcal{I}}, \\ \neg C^{\mathcal{I}} &= \Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}, \\ (C \sqcap D)^{\mathcal{I}} &= C^{\mathcal{I}} \cap D^{\mathcal{I}}, \\ (C \sqcup D)^{\mathcal{I}} &= C^{\mathcal{I}} \cup D^{\mathcal{I}}, \\ (\exists r.C)^{\mathcal{I}} &= \{d \in \Delta^{\mathcal{I}} \mid \text{there exists } (d, e) \in r^{\mathcal{I}} \text{ with } e \in C^{\mathcal{I}}\}, \\ (\forall r.C)^{\mathcal{I}} &= \{d \in \Delta^{\mathcal{I}} \mid \text{for all } (d, e) \in r^{\mathcal{I}} \text{ we have } e \in C^{\mathcal{I}}\}. \end{aligned}$$

acronym	syntax	semantics
$\mathcal{I}$	$\exists r^-.C$	$\{d \in \Delta^{\mathcal{I}} \mid \text{there exists } e \in C^{\mathcal{I}} : (e, d) \in r^{\mathcal{I}}\}$
$\mathcal{O}$	$\{a\}$	$\{a^{\mathcal{I}}\}$
$\mathcal{F}$	$(\leq 1 \ r)$	$\{d \in \Delta^{\mathcal{I}} \mid \text{there is at most one } e \in \Delta^{\mathcal{I}} \text{ with } (d, e) \in r^{\mathcal{I}}\}$
$\mathcal{S}$	$\text{trans}(r)$	$r^{\mathcal{I}} \text{ is transitive}$
$\mathcal{H}$	$r \sqsubseteq s$	$r^{\mathcal{I}} \subseteq s^{\mathcal{I}}$

■ **Table 1** Extensions of $\mathcal{ALC}$

Let $\mathcal{O}$ be an ontology and $\mathcal{I}$ be an interpretation. Then $\mathcal{I}$ *satisfies* a CI $C \sqsubseteq D$ if $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$, and $\mathcal{I}$ is a *model* of $\mathcal{O}$ if it satisfies all CIs in $\mathcal{O}$. The ontology $\mathcal{O}$ *entails* a CI $C \sqsubseteq D$, in symbols $\mathcal{O} \models C \sqsubseteq D$, if every model of $\mathcal{O}$ satisfies $C \sqsubseteq D$; it *entails* another ontology $\mathcal{O}'$, written $\mathcal{O} \models \mathcal{O}'$ if $\mathcal{O} \models C \sqsubseteq D$ for every CI $C \sqsubseteq D \in \mathcal{O}'$. In case $\mathcal{O} \models C \sqsubseteq D$, we also say that C is *subsumed by* D under $\mathcal{O}$. We drop $\mathcal{O}$ if it is empty and write $\models C \sqsubseteq D$ for $\emptyset \models C \sqsubseteq D$.

A *signature* Σ is a finite set of concept, role, and individual names, uniformly referred to as *symbols*. We use $\text{sig}(X)$ to denote the set of symbols used in any syntactic object X such as a concept or an ontology. An $\mathcal{ALC}(\Sigma)$ *concept* is an $\mathcal{ALC}$ concept C with $\text{sig}(C) \subseteq \Sigma$. The *size* of a (finite) syntactic object X , denoted $\|X\|$, is the number of symbols needed to represent it as a word. The *role depth* $\text{rd}(C)$ of a concept C is the maximal nesting depth of existential and universal restrictions in C .

Extensions and Restrictions of $\mathcal{ALC}$. Depending on the application, one requires more or less expressive power to describe the domain knowledge, which is why different extensions and restrictions of $\mathcal{ALC}$ have been investigated. The most prominent restriction is certainly $\mathcal{EL}$, in which only the concept constructors $\top$, A , $C \sqcap D$, $\exists r.C$ are allowed (and thus no negation $\neg C$, disjunction $C \sqcup D$, and universal restriction $\forall r.C$) [10]. We also consider several important extensions, see [11] for more details on these. *Inverse roles* (abbreviated with the acronym $\mathcal{I}$) take the form r^- for a role name r . They are interpreted as the inverse of the interpretation of the role, that is, $(r^-)^{\mathcal{I}} = \{(e, d) \mid (d, e) \in r^{\mathcal{I}}\}$ and can be used to travel edges in the converse direction. *Nominals* (acronym $\mathcal{O}$) take the form $\{a\}$ for individual names $a \in \mathbf{N}_i$ and are used to describe singleton concepts. *(Local) functionality restrictions* (acronym $\mathcal{F}$) take the form $(\leq 1 \ r)$ and describe the set of all individuals that have at most one r -successor. *Role hierarchies* (acronym $\mathcal{H}$) take the form $r \sqsubseteq s$ and can be used to describe relations between roles. Finally, some of the roles may be declared as *transitive* (acronym $\mathcal{S}$), which means that they have to be interpreted as transitive relations. Table 1 displays an overview of the extensions and their semantics. The names of the extensions are obtained in a canonical way by appending the acronym of the additional constructor, e.g., $\mathcal{ALCHI}$ is the extension of $\mathcal{ALC}$ with role hierarchies and inverse roles. A bit of care has to

8 Interpolation in Knowledge Representation

be taken when combining certain features, e.g., transitive roles with functionality restrictions; we adopt the standard restrictions known from the literature [11].

► **Example 2.** The example from the introduction can be represented as follows as an $\mathcal{ALC}$ ontology. The first and last axioms are also in $\mathcal{EL}$.

$$\begin{aligned} \text{Car} &\sqsubseteq \exists \text{hasPart.PrimeMover} & \text{PrimeMover} &\sqsubseteq \text{DieselEngine} \sqcup \text{GasEngine} \sqcup \text{ElectricMotor} \\ \text{Car} &\sqcap \exists \text{hasPart.ElectricMotor} & &\sqsubseteq \text{ElectricCar} \end{aligned}$$

In $\mathcal{ALCFIO}$, we can additionally express that every engine belongs to at most one car, and that a German car is a car built in Germany.

$$\text{PrimeMover} \sqsubseteq (\leq 1 \text{ hasPart}^-) \quad \text{GermanCar} \equiv \text{Car} \sqcap \exists \text{madeIn}.\{\text{Germany}\} \quad \lrcorner$$

Model Theory. We next recall the model-theoretic notion of a bisimulation, which is very useful in the context of interpolation in DLs. Let Σ be a signature and $\mathcal{I}_1, \mathcal{I}_2$ be interpretations. A relation $Z \subseteq \Delta^{\mathcal{I}_1} \times \Delta^{\mathcal{I}_2}$ is an $\mathcal{ALC}(\Sigma)$ -bisimulation between $\mathcal{I}_1$ and $\mathcal{I}_2$ if the following conditions are satisfied for all $(d, e) \in Z$:

Atom for all concept names $A \in \Sigma$: $d \in A^{\mathcal{I}_1}$ iff $e \in A^{\mathcal{I}_2}$,

Back for all role names $r \in \Sigma$ and all $(d, d') \in r^{\mathcal{I}_1}$, there is $(e, e') \in r^{\mathcal{I}_2}$ such that $(d', e') \in Z$,

Forth for all role names $r \in \Sigma$ and all $(e, e') \in r^{\mathcal{I}_2}$, there is $(d, d') \in r^{\mathcal{I}_1}$ such that $(d', e') \in Z$.

A *pointed interpretation* is a pair $\mathcal{I}, d$ with $\mathcal{I}$ an interpretation and $d \in \Delta^{\mathcal{I}}$. For pointed interpretations $\mathcal{I}_1, d_1$ and $\mathcal{I}_2, d_2$, we write $\mathcal{I}_1, d_1 \sim_{\mathcal{ALC}, \Sigma} \mathcal{I}_2, d_2$ in case $\mathcal{I}_1, d_1$ and $\mathcal{I}_2, d_2$ are $\mathcal{ALC}(\Sigma)$ -bisimilar, that is, if there is an $\mathcal{ALC}(\Sigma)$ -bisimulation Z between $\mathcal{I}_1$ and $\mathcal{I}_2$ with $(d_1, d_2) \in Z$. Bisimulations are a powerful tool since they capture the expressive power of $\mathcal{ALC}$. Indeed, if $\mathcal{I}_1, d_1 \sim_{\mathcal{ALC}, \Sigma} \mathcal{I}_2, d_2$, then d_1 and d_2 satisfy the same $\mathcal{ALC}(\Sigma)$ concepts, that is, $d_1 \in C^{\mathcal{I}_1}$ iff $d_2 \in C^{\mathcal{I}_2}$, for all $\mathcal{ALC}(\Sigma)$ concepts. The converse direction does not hold in general, but in relevant cases (for example, when $\mathcal{I}_1$ and $\mathcal{I}_2$ are finite). We refer the interested reader to [53] for a more detailed account on model theory.

For the purpose of this chapter, it is worth mentioning that for each extension/restriction $\mathcal{L}$ of $\mathcal{ALC}$ there is a corresponding notion of bisimulation, denoted $\sim_{\mathcal{L}, \Sigma}$, that captures the expressive power of $\mathcal{L}$. As a concrete example, an $\mathcal{ALCO}(\Sigma)$ -bisimulation Z between $\mathcal{I}_1$ and $\mathcal{I}_2$ is an $\mathcal{ALC}(\Sigma)$ -bisimulation that additionally satisfies the following for all $(d, e) \in Z$:

AtomI for all individual names $a \in \Sigma$: $d = a^{\mathcal{I}_1}$ iff $e = a^{\mathcal{I}_2}$.

3 Uniform Interpolation for Description Logic Ontologies

The central notion we are concerned with in this section is the following adaptation of Definition 1 to description logic ontologies.

► **Definition 3** (Uniform interpolant). *Let $\mathcal{L}$ be a DL, $\mathcal{O}$ an $\mathcal{L}$ ontology, and Σ a signature. Then, an $\mathcal{L}$ ontology $\mathcal{O}^\Sigma$ is called uniform $\mathcal{L}(\Sigma)$ -interpolant of $\mathcal{O}$ if*

1. $\mathcal{O} \models \mathcal{O}^\Sigma$,
2. $\text{sig}(\mathcal{O}^\Sigma) \subseteq \Sigma$,
3. *for every $\mathcal{L}$ CI α such that $\text{sig}(\alpha) \subseteq \Sigma$ and $\mathcal{O} \models \alpha$, also $\mathcal{O}^\Sigma \models \alpha$.*

Conditions 1 and 3 imply that $\mathcal{O}$ and a uniform $\mathcal{L}(\Sigma)$ -interpolant $\mathcal{O}^\Sigma$ of $\mathcal{O}$ entail precisely the same $\mathcal{L}(\Sigma)$ concept inclusions. This latter condition is called $\mathcal{L}(\Sigma)$ -inseparability and of independent interest.

► **Definition 4** (Inseparability). *Let $\mathcal{O}_1, \mathcal{O}_2$ be $\mathcal{L}$ ontologies and Σ a signature. Then, $\mathcal{O}_1$ and $\mathcal{O}_2$ are $\mathcal{L}(\Sigma)$ -inseparable, in symbols $\mathcal{O}_1 \equiv_\Sigma^\mathcal{L} \mathcal{O}_2$, if $\mathcal{O}_1 \models \alpha$ iff $\mathcal{O}_2 \models \alpha$ for every $\mathcal{L}(\Sigma)$ CI α .*

It is immediate from Definition 3 that uniform interpolants are unique modulo logical equivalence, which is why we often speak of *the* uniform $\mathcal{L}(\Sigma)$ -interpolant of $\mathcal{O}$. In modal logic terminology, this form of interpolation could be classified as *turnstile* interpolation for global consequence. It is *uniform* in the sense that the interpolant has to work for all $\mathcal{L}(\Sigma)$ ontologies that are a consequence of $\mathcal{O}$; we refer the reader to Chapter 9 for more on uniform interpolation. To illustrate the notion, consider the following example [96, Example 2].

► **Example 5.** Consider the ontology $\mathcal{O}$ consisting of the following CIs:

$$\text{Uni} \sqsubseteq \exists \text{hasEnrolled}.\text{Grad} \sqcap \exists \text{hasEnrolled}.\text{Undergrad} \quad (4)$$

$$\text{Grad} \sqsubseteq \neg \text{Undergrad} \quad (5)$$

$$\text{Uni} \sqsubseteq \neg \text{Grad} \quad (6)$$

$$\text{Uni} \sqsubseteq \neg \text{Undergrad} \quad (7)$$

Then, it can be verified that the ontology $\mathcal{O}'$ consisting of CI (7) and the CI

$$\text{Uni} \sqsubseteq \exists \text{hasEnrolled}.\text{Undergrad} \sqcap \exists \text{hasEnrolled}.\neg \text{Undergrad} \sqcap \neg \text{Uni}$$

is a uniform $\mathcal{ALC}(\Sigma)$ -interpolant of $\mathcal{O}$ for $\Sigma = \{\text{Uni}, \text{hasEnrolled}, \text{Undergrad}\}$. Sometimes $\mathcal{O}'$ is called the result of *forgetting* the concept name *Grad* in $\mathcal{O}$, since $\mathcal{O}'$ contains exactly the same information about the signature $\Sigma = \text{sig}(\mathcal{O}) \setminus \{\text{Grad}\}$. $\lrcorner$

Unfortunately, the existence of uniform interpolants is by no means guaranteed, as the following example illustrates already for very simple DL ontologies.

► **Example 6.** Consider the $\mathcal{EL}$ ontology $\mathcal{O} = \{A \sqsubseteq B, B \sqsubseteq \exists r.B\}$ and $\Sigma = \{A, r\}$. A uniform $\mathcal{EL}(\Sigma)$ -interpolant would need to entail precisely the CIs

$$A \sqsubseteq \exists r.\top, \quad A \sqsubseteq \exists r.\exists r.\top, \quad A \sqsubseteq \exists r.\exists r.\exists r.\top, \quad \dots$$

10 Interpolation in Knowledge Representation

which is not possible for a finite set of $\mathcal{EL}$ or $\mathcal{ALC}$ CIs. (Actually, there is not even first-order sentence that entails precisely these CIs, c.f. Chapter 2 for uniform interpolation in first-order logic.) The reader may conjecture that the lack of a uniform interpolant is due to cyclicity in $\mathcal{O}$, but this is not the entire story. Consider the $\mathcal{EL}$ ontology

$$\mathcal{O} = \{ \quad A \sqsubseteq \exists r.B, \quad A_0 \sqsubseteq \exists r.(A_1 \sqcap B), \quad E \equiv A_1 \sqcap B \sqcap \exists r.(A_2 \sqcap B) \quad \}$$

and signature $\Sigma = \{A, A_0, A_1, r, E\}$. Reference [73] classifies $\mathcal{O}$ as Σ -loop free and shows that hence $\mathcal{O}$ has a uniform $\mathcal{EL}(\Sigma)$ -interpolant. However, $\mathcal{O}$ fails to have a uniform $\mathcal{ALC}(\Sigma)$ -interpolant [96, Example 3]. $\lrcorner$

As we have argued in the introduction, uniform interpolants are useful in a range of applications in KR. Since they do not always exist, it is interesting from a theoretical point of view to investigate existence as a decision problem. From a practical perspective, it is interesting to compute them (in case they exist), study their size, and to deal with the fact that they may not exist. We will address exactly these questions. More precisely, in Section 3.1, we discuss the complexity of the existence problem and the size of uniform interpolants. In Section 3.2, we discuss methods to compute uniform interpolants in practice, addressing also the question what to do if they do not exist. Finally, in Section 3.3, we discuss further related notions that are relevant to DL specific applications. Throughout the section, we focus on the DLs $\mathcal{EL}$ and $\mathcal{ALC}$, as these are the ones for which most results are known.

3.1 Existence and Size

As motivated in the previous section, the focus of this section will be the following *uniform interpolant existence problem* for DL $\mathcal{L}$:

Input $\mathcal{L}$ ontology $\mathcal{O}$, signature Σ .

Question Does there exist a uniform $\mathcal{L}(\Sigma)$ -interpolant of $\mathcal{O}$?

The main result known here is that this problem is decidable for both $\mathcal{EL}$ and $\mathcal{ALC}$, but has complexity one exponential higher than standard reasoning tasks in these DLs.

► **Theorem 7** ([92, 96]). *Uniform interpolant existence is EXPTIME-complete for $\mathcal{EL}$ and 2EXPTIME-complete for $\mathcal{ALC}$.*

We are interested in the size of the uniform interpolants, if they exist, and tight bounds are known here as well. Interestingly, while the complexity of deciding existence is higher for $\mathcal{ALC}$ than for $\mathcal{EL}$, they exhibit the same bounds on the size of uniform interpolants.

► **Theorem 8** ([92, 96, 103]). *Let $\mathcal{L}$ be either $\mathcal{EL}$ or $\mathcal{ALC}$.*

1. *If some $\mathcal{L}$ ontology $\mathcal{O}$ has a uniform $\mathcal{L}(\Sigma)$ -interpolant for signature Σ , then there is one of size at most triple exponential in the size of $\mathcal{O}$.*

2. *There is a family $\mathcal{O}_n$, $n \geq 1$ of $\mathcal{L}$ ontologies and a signature Σ such that the size of $\mathcal{O}_n$ is polynomial in n , a uniform $\mathcal{L}(\Sigma)$ -interpolant exists for all $n \geq 1$, but any uniform $\mathcal{L}(\Sigma)$ -interpolant has size at least triple exponential in n .*

It is beyond the scope of this chapter to present all proof details of these results, but we shall give some intuition on how they are obtained. We start with the size lower bounds, that is, Point 2 of Theorem 8, by recalling the family of $\mathcal{EL}$ ontologies from [103] that is used to prove this result. Let $n \geq 1$. Then $\mathcal{O}_n$ consists of the following CIs:

$$A_1 \sqsubseteq \overline{X_1} \sqcap \dots \sqcap \overline{X_n} \qquad A_2 \sqsubseteq \overline{X_1} \sqcap \dots \sqcap \overline{X_n} \qquad (8)$$

$$\prod_{\sigma \in \{r,s\}} \exists \sigma. (\overline{X_i} \sqcap \overline{X_j}) \sqsubseteq \overline{X_i} \qquad \prod_{\sigma \in \{r,s\}} \exists \sigma. (X_i \sqcap \overline{X_j}) \sqsubseteq X_i \qquad 1 \leq j < i \leq n \qquad (9)$$

$$\prod_{\sigma \in \{r,s\}} \exists \sigma. (\overline{X_i} \sqcap X_{i-1} \sqcap \dots \sqcap X_1) \sqsubseteq X_i \qquad 1 \leq i \leq n \qquad (10)$$

$$\prod_{\sigma \in \{r,s\}} \exists \sigma. (X_i \sqcap X_{i-1} \sqcap \dots \sqcap X_1) \sqsubseteq \overline{X_i} \qquad 1 \leq i \leq n \qquad (11)$$

$$X_0 \sqcap \dots \sqcap X_n \sqsubseteq B \qquad (12)$$

Intuitively, $\mathcal{O}_n$ constructs an n -bit binary counter via the concept names $X_i, \overline{X_i}$ for $1 \leq i \leq n$. The satisfaction of these names encodes a number between 0 and $2^n - 1$ using n bits: satisfaction of X_i means that the i th bit has value 1, and $\overline{X_i}$ represents a value of 0 at position i . Using this, elements can be assigned a counter value k . Specifically, the CIs in (8) make sure that instances of A_1 and A_2 have a counter value of 0 (all bits are 0). CIs (9)–(11) make sure that, if both an r and an s -successor of an element d has a counter value of k , then d has a counter value of $k + 1$. This is done by specifying how the bits on d should be set depending on the bits in the successors. Specifically, the CIs in (9) describe the situation where the bit value at position i should remain the same (at least one lower bit in the successor has value 0). CIs (10) and (11) describe that a bit at position i should flip if all lower bits in the successor have a value of 1. Finally, the CI (12) states that elements with a counter value of 2^{n-1} (all bits are 1) must satisfy B .

This construction has the following effect: any element d that is the root of a full binary r/s -tree of depth 2^n whose leaves satisfy A_1 or A_2 , will be an instance of B . For $\Sigma = \{A_1, A_2, B, r, s\}$, any uniform Σ -interpolant will have to preserve this behaviour without using the counter encoded with the concept names $X_i, \overline{X_i}$. To construct the uniform $\mathcal{EL}(\Sigma)$ -interpolant for $\mathcal{O}_n$, we define inductively sets $\mathcal{C}_i$ of concepts by setting $\mathcal{C}_0 = \{A_1, A_2\}$ and $\mathcal{C}_{i+1} = \{\exists r.C \sqcap \exists s.C \mid C \in \mathcal{C}_i\}$ for $i \geq 0$. It is not hard to see that the cardinality of each $\mathcal{C}_i$ is 2^{2^i} . Moreover, for each $C \in \mathcal{C}_{2^n}$, $\mathcal{O}_n \models C \sqsubseteq B$, and we cannot capture these entailments more concisely than by adding all those $C \sqsubseteq B$ to the uniform interpolant. It follows that the uniform interpolant contains at least $2^{2^{2^n}}$ CIs, and is thus of size triple exponential in n .

This finishes the proof sketch for $\mathcal{EL}$. The same construction does not directly yield a triple-exponential lower bound for $\mathcal{ALC}$, since we can here use the single concept $A_1 \sqcup A_2$ in

12 Interpolation in Knowledge Representation

case of the two concepts in $\mathcal{C}_0$, which reduces the size of the uniform interpolant to double exponential. The triple exponential lower bound on the size of uniform interpolants for $\mathcal{ALC}$ from [96, 47], is shown similarly, but relies on a 2^n -bit counter instead of an n -bit counter.

We now return to the existence problem, this time concentrating on $\mathcal{ALC}$. Let us fix an $\mathcal{ALC}$ ontology $\mathcal{O}$ and a signature Σ as input. We first give a syntactic characterization for the existence of a uniform interpolant. For $n \geq 0$, define $\mathcal{O}_n^\Sigma$ as the set of all $\mathcal{ALC}(\Sigma)$ CIs entailed by $\mathcal{O}$ whose role depth is at most n . If there exists a uniform interpolant, it must be equivalent to $\mathcal{O}_n^\Sigma$ for some n , since all its CIs would be contained in $\mathcal{O}_n^\Sigma$, and conversely $\mathcal{O} \models \mathcal{O}_n^\Sigma$ by construction. This means that non-existence of uniform interpolants implies that for every n , there exists some $k > n$ such that $\mathcal{O}_n^\Sigma \not\models \mathcal{O}_k^\Sigma$.

We characterize when some $\mathcal{O}_n^\Sigma$ is (not) a uniform interpolant using tree interpretations, which are interpretations $\mathcal{I}$ for which the relation $\bigcup \{r^{\mathcal{I}} \mid r \in \mathbf{N}_R\}$ forms a directed tree. The root of such an interpretation is then the root of that directed tree. For an integer m , we use $\mathcal{I}^m$ to refer to the interpretation obtained by removing all individuals which are not reachable from the root via a role path of length at most m . We then have the following lemma.

► **Lemma 9.** *Let $\mathcal{O}$ be an ontology, Σ a signature and $n > 0$. Then, $\mathcal{O}_n^\Sigma$ is not a uniform $\mathcal{ALC}(\Sigma)$ -interpolant of $\mathcal{O}$ if there are tree interpretations $\mathcal{I}_1, \mathcal{I}_2$ with root d such that*

1. $\mathcal{I}_1^n = \mathcal{I}_2^n$,
2. $\mathcal{I}_1, d \sim_{\mathcal{ALC}, \Sigma} \mathcal{J}, d'$ for some model $\mathcal{J}$ of $\mathcal{O}$ and $d' \in \Delta^{\mathcal{J}}$,
3. $\mathcal{I}_2, d \not\sim_{\mathcal{ALC}, \Sigma} \mathcal{J}, d'$ for all models $\mathcal{J}$ of $\mathcal{O}$ and $d' \in \Delta^{\mathcal{J}}$, and
4. for all role successors e of d , $\mathcal{I}_2, e \sim_{\mathcal{ALC}, \Sigma} \mathcal{J}, d'$ for some model $\mathcal{J}$ of $\mathcal{O}$ and $d' \in \Delta^{\mathcal{J}}$.

Intuitively, these conditions express that one has to look at a role depth beyond n to capture all Σ entailments of $\mathcal{O}$, or, equivalently, to capture all pointed interpretations that are Σ -bisimilar to some pointed model of $\mathcal{O}$. Together with Condition 2, Condition 1 ensures that the root individual in both pointed interpretations satisfies the same CIs up to role depth n , which means they satisfy all CIs in $\mathcal{O}_n^\Sigma$. However, $\mathcal{I}_2, d$ is not Σ -bisimilar to all pointed models of $\mathcal{O}$, while $\mathcal{I}_1$ is (Condition 3). Finally, Condition 4 states that the root is the point where the bisimulation breaks, since in $\mathcal{I}_2$, all successors of d are Σ -bisimilar to pointed models of $\mathcal{O}$ as required. In other words, $\mathcal{I}_2, d$ witnesses precisely that $\mathcal{O}_n^\Sigma$ is not a uniform Σ -interpolant of $\mathcal{O}$.

Set $M_{\mathcal{O}} = 2^{2 \cdot 2^{\|\mathcal{O}\|}} + 1$. We can show that if Lemma 9 applies for $n = M_{\mathcal{O}}$, then it also applies for all $n > M_{\mathcal{O}}$, meaning that then there does not exist a uniform $\mathcal{ALC}(\Sigma)$ -interpolant. To do this, we use a construction based on types. Let Γ denote the set of all subconcepts that occur in $\mathcal{O}$, closed under single negation. Given an interpretation $\mathcal{I}$ and some individual $d \in \Delta^{\mathcal{I}}$, we then define the *type* $tp_{\mathcal{I}}(d)$ of d in $\mathcal{I}$ as

$$tp_{\mathcal{I}}(d) = \{C \in \Gamma \mid d \in C^{\mathcal{I}}\}.$$

To capture the types of elements that are bisimilar to d , we define the *extension set* of d as

$$\text{Ext}(d, \mathcal{I}) = \{\text{tp}_{\mathcal{J}}(d') \mid \mathcal{J} \models \mathcal{O}, d' \in \Delta^{\mathcal{J}} \text{ and } \mathcal{J}, d' \sim_{\Sigma} \mathcal{I}, d\}$$

There are at most $2^{2^{\|\mathcal{O}\|}}$ extension sets. Now assume Lemma 9 applies for $n = M_{\mathcal{O}}$, and let $\mathcal{I}_1$ and $\mathcal{I}_2$ be the witnessing tree interpretations. We have $\mathcal{I}_1^n = \mathcal{I}_2^n$, and now look at the successors in the leafs of $\mathcal{I}_1^n$, i.e. the elements in $\mathcal{I}_1^n$ without a role successor. Specifically, for such a pair, we define the set $D_n(\mathcal{I}_1, \mathcal{I}_2)$ to contain all leafs in $\mathcal{I}_1^n$ which differ in $\mathcal{I}_1^{n+1}$ and $\mathcal{I}_2^{n+1}$ in their sets of role successors, by which we mean that they either have a different set of role successors or some role successor that satisfies a different set of concept names in one interpretation compared to the other. Fix a pair $\mathcal{I}_1, \mathcal{I}_2$ of interpretations that satisfy the conditions in Lemma 9 for $n = M_{\mathcal{O}}$. If $D_n(\mathcal{I}_1, \mathcal{I}_2) = \emptyset$, then lemma also applies for $n = M_{\mathcal{O}} + 1$, and we are done. Otherwise, we pick some $f \in D_n(\mathcal{I}_1, \mathcal{I}_2)$, and consider the path $d = e_1, e_2, \dots, e_n = f$ of elements such that, for $1 \leq i < n$, $\langle e_i, e_{i+1} \rangle \in r_i^{\mathcal{I}_1}$ for some role name r_i . Because $n = M_{\mathcal{O}} = 2^{2 \cdot 2^{\|\mathcal{O}\|}} + 1$, and there are at most $(2^{2^{\|\mathcal{O}\|}})^2 = 2^{2 \cdot 2^{\|\mathcal{O}\|}}$ pairs of extension sets, there must be some $1 \leq i < j \leq n$ such that $\text{Ext}(e_i, \mathcal{I}_1) = \text{Ext}(e_j, \mathcal{I}_1)$ and $\text{Ext}(e_i, \mathcal{I}_2) = \text{Ext}(e_j, \mathcal{I}_2)$. In both $\mathcal{I}_1$ and $\mathcal{I}_2$, we now replace the subtree below e_j with a copy of the subtree below e_i , resulting in two new interpretations $\mathcal{K}_1$ and $\mathcal{K}_2$. One can show that this construction preserves the conditions in Lemma 9, so that $\mathcal{K}_1$ and $\mathcal{K}_2$ also witness it. In addition, we have $D_n(\mathcal{K}_1, \mathcal{K}_2) \subsetneq D_n(\mathcal{I}_1, \mathcal{I}_2)$. We can repeat this operation until we obtain a pair $\mathcal{J}_1, \mathcal{J}_2$ of interpretations for which $D_n(\mathcal{J}_1, \mathcal{J}_2) = \emptyset$, and thus $\mathcal{J}_1^{n+1} = \mathcal{J}_2^{n+1}$, and obtain that Lemma 9 also holds for $n + 1$. This gives us the following lemma.

► **Lemma 10.** *There is a uniform $\mathcal{L}(\Sigma)$ -interpolant of $\mathcal{O}$ iff $\mathcal{O}_n^{\Sigma} \models \mathcal{O}_{n+1}^{\Sigma}$ for $n = M_{\mathcal{O}}$.*

A consequence of lemma 10 is that if a uniform interpolant exists, then $\mathcal{O}_n^{\Sigma}$ is one, where $n = M_{\mathcal{O}}$. Thus, it gives a bound on the role depth of uniform interpolants. Since, up to equivalence, there are only finitely many $\mathcal{ALC}(\Sigma)$ CIs of bounded role depth, this also shows that the existence problem is decidable. Unfortunately, the bounds we get in this way are non-elementary. To establish the 2EXPTIME-upper bound claimed in Theorem 7, [96] uses an automata-based approach to decide existence of uniform interpolants. The idea is to construct a tree automaton that decides the existence of interpretations $\mathcal{I}_1$ and $\mathcal{I}_2$ that satisfy the conditions in Lemma 9 for $n = M_{\mathcal{O}}$. In the same paper, they show the triple exponential upper bound on the size of uniform $\mathcal{ALC}$ -interpolants claimed in Point 1 of Theorem 8. The proof is by a reduction of the problem of computing uniform interpolants of ontologies to the problem of computing uniform interpolants of *concepts* and apply known results for the latter [118]. For the complexity lower bounds, we refer the reader to [96] as well.

Interestingly, the strategy for deciding existence of uniform interpolants for $\mathcal{EL}$ is similar [92]: it relies on a characterization in the style of Lemma 9 (using simulations instead of bisimulations), and shows a bound on the role depth of uniform interpolants in style of Lemma 10 (which is single exponential in this case). Automata are then used as well, though in a different way, to obtain the complexity upper bounds claimed in Theorem 7. The size bounds for $\mathcal{EL}$ in Theorem 8 are shown in [103].

14 Interpolation in Knowledge Representation

Name of Tool	Logic	Technique	Dealing with Non-Existence
NUI [73]	$\mathcal{EL}$	TBox unfolding	–
- [89]	$\mathcal{ALC}$	resolution	approximation
LETHE [76]	$\mathcal{ALC}$	resolution + Ackermann’s Lemma	fixpoints/auxiliary symbols
FAME [131]	$\mathcal{ALC}$	Ackermann’s Lemma	fixpoints/auxiliary symbols

■ **Table 2** Overview of tools for computing uniform interpolants of ontologies in practice.

3.2 Computing Uniform Interpolants in Practice

Despite these high complexity bounds, algorithms for computing uniform interpolants have been described for many description logics, ranging from light-weight DLs such as DL-Lite [127] and $\mathcal{EL}$ [73, 90, 88] to $\mathcal{ALC}$ and more expressive DLs [128, 89, 76, 131, 75]. Many of these were implemented (see Table 2), and are able to compute results even for larger realistic ontologies. In this section, we discuss how implemented methods compute uniform interpolants of $\mathcal{ALC}$ ontologies, describing the method used by LETHE in detail.

The first practical issue is of course that uniform interpolants do not always exist. In many cases, a pragmatic solution could be to just extend the desired signature by the problematic symbols. For the case where this is not an option, there are three solutions, all of which are implemented by some tools.

Option 1: We approximate the uniform interpolant up to a given role depth, which might be sufficient if we are only interested in entailments of bounded role depth. The resolution based approach proposed in [89] provides guarantees on the role depth of preserved entailments—unfortunately these guarantees require an exponential increase in the role depth in the uniform interpolant.

Option 2: We extend the DL by *greatest fixpoint operators*. As argued above, for the ontology $\mathcal{O} = \{A \sqsubseteq B, B \sqsubseteq \exists r.B\}$, a uniform $\mathcal{ALC}(\{A, r\})$ -interpolant does not exist. However, there does exist one in $\mathcal{ALC}\mu$: $\mathcal{O}_\Sigma = \{A \sqsubseteq \nu X. \exists r.X\}$. Indeed, $\mathcal{ALC}\mu$ does have the uniform interpolation property, which follows from the corresponding result for the modal μ -calculus [26]; see also Chapter 11. $\mathcal{ALC}\mu$ extends $\mathcal{ALC}$ by concepts of the form $\nu X.C[X]$, where X is taken from a set N_V of *concept variables* that is pair-wise disjoint with N_C , N_R and N_I , and $C[X]$ is a concept in which X is used like a concept name, but occurs only positively, that is, under an even number of negations. For a formal definition of the semantics, we refer to [23]. Intuitively, $\nu X.C[X]$ corresponds to the limit of the sequence

$$\top, \quad C[\top], \quad C[C[\top]], \quad C[C[C[\top]]], \dots,$$

where in each case, $C[D]$ refers to the result of replacing X in $C[X]$ by D .

Option 3: Since fixpoint operators are harder to understand for end-users and also not supported by the OWL standard, which is the format used to store ontologies in practical applications, a third option is to simulate greatest fixpoint operators using auxiliary

concepts: In particular, we can replace any greatest fixpoint expression $\nu X.C[X]$, provided that it occurs only positively (e.g., under an even number of negations on the left-hand side of a CI, and under an odd number of negations on the right-hand side), by a fresh concept name D for which we add a new CI $D \sqsubseteq C[D]$. We then approximate the uniform Σ -interpolant *signature-wise*, resulting in an ontology that is Σ -inseparable from the original ontology and uses names outside of Σ in a syntactically restricted way.

Regardless of the option chosen, the general idea of practical uniform interpolation methods is to eliminate the names that should not occur in the interpolant using dedicated inferences. For this, we find two approaches: 1) using resolution [89] (as in Chapter 1 and Chapter 12)) or 2) using Ackermann's Lemma [1, 131] (also discussed in Chapter 12). The method in [76], which we present in the following, uses both in combination.

Because $\mathcal{ALC}$ is more complex than propositional logics, and syntactically more restricted than first-order logics, in order to pursue 1), we cannot use standard resolution methods, but need a specific resolution procedure that is appropriate for the logic and the aim of computing uniform interpolants. The first resolution-based approach for uniform interpolation in $\mathcal{ALC}$ was presented in [89] and is based on a complex inference system for modal logic introduced in [31] and extended for uniform interpolation in modal logics in [56]. This inference system intuitively specifies an unbounded number of inference rules that allow to perform resolution on symbols occurring in different nesting levels of role restrictions inside a CI. By increasing the bound on the nesting level on which inferences are performed, we can then approximate the uniform interpolant up to a specified role depth.

Ackermann's Lemma [1] has been extensively used in the context of Second-Order Quantifier Elimination. To formulate it for DLs, we use a stronger version of inseparability as introduced in Definition 4, in which instead of $\mathcal{L}$ concept inclusions arbitrary second-order logic (SOL) sentences are considered. We use $\mathcal{O}_1 \equiv_{\Sigma}^{\text{SOL}} \mathcal{O}_2$ to denote that $\mathcal{O}_1, \mathcal{O}_2$ entail the same SOL sentences over signature Σ . Let $\mathcal{O}[A \mapsto C]$ denote the ontology that is obtained from $\mathcal{O}$ by replacing every occurrence of A by C .

► **Lemma 11.** *Let $\mathcal{O}$ be an ontology, A a concept name, C a concept with $A \notin \text{sig}(C)$, and $\Sigma = \text{sig}(\mathcal{O}) \setminus \{A\}$. Assume A occurs only positively in $\mathcal{O}$. Then, $\mathcal{O} \cup \{A \sqsubseteq C\} \equiv_{\Sigma}^{\text{SOL}} \mathcal{O}[A \mapsto C]$.*

Intuitively, this means that, if we can bring the ontology into the right form, we can compute a very strong kind of uniform interpolants by simply applying the equivalence in Lemma 11 left-to-right. If A does occur on the right-hand side in $A \sqsubseteq C$, we can use a generalization of Ackermann's lemma from [104] that introduces greatest fixpoints. When restricted to DLs, it can be formulated as follows.¹

¹ For the original FO formalization of both lemmas and their application to uniform interpolation, see also Chapter 12.

16 Interpolation in Knowledge Representation

► **Lemma 12** (Generalized Ackermann’s Lemma). *Let $\mathcal{O}$ be an ontology, A a concept name, $C[A]$ a concept in which A occurs, and $\Sigma = \text{sig}(\mathcal{O}) \setminus \{A\}$. Assume A occurs only positively in $\mathcal{O}$ and $C[A]$. Then, the following holds:*

$$\mathcal{O} \cup \{A \sqsubseteq C[A]\} \equiv_{\Sigma}^{\text{SOL}} \mathcal{O}[A \mapsto \nu X.C[X]]$$

For both lemmas to be applicable, we have to bring the ontology into the required form. In particular, one has to isolate all negative occurrences of A into a single axiom of the form $A \sqsubseteq C$, which is not always possible. The approach in [131], implemented in the uniform interpolation tool FAME, essentially follows this strategy, and uses additional constructs such as inverse roles to make this possible, for instance, using the equivalence of axioms $C \sqsubseteq \forall r.D$ and $\exists r^{-}.C \sqsubseteq D$. The full method implemented in FAME is much more complex, and cannot only eliminate concept names, but also role names. The advantage of using (generalized) Ackermann’s lemma is that the computed uniform interpolants are of a very strong kind, namely they are $\text{SOL}(\Sigma)$ -inseparable. This is why the method implemented in FAME is also called *semantic forgetting*. Since $\text{SOL}(\Sigma)$ -inseparability is undecidable (see Section 3.3), it is in general not possible to perform semantic forgetting to obtain an ontology expressed in a decidable logic, and indeed FAME cannot compute uniform interpolants for all possible inputs.

The technique in [76], implemented in the uniform interpolation tool LETHE, combines resolution with Ackermann’s lemma and is based on a special normal form for $\mathcal{ALC}$ ontologies. In this normal form, axioms are represented as disjunctions $L_1 \sqcup \dots \sqcup L_n$ of literals of the following forms, where $A \in \mathbf{N}_C$ and D belongs to special set $\mathbf{N}_D \subseteq \mathbf{N}_C$ of concept names called *definers*:

$$A \quad | \quad \neg A \quad | \quad \exists r.D \quad | \quad \forall r.D$$

Such disjunctions are called *DL clauses*, and are interpreted *globally*. For example, the concept inclusion $A \sqsubseteq B$ is represented as the DL clause $\neg A \sqcup B$. $\mathcal{ALC}$ ontologies $\mathcal{O}$ are normalized into $\text{sig}(\mathcal{O})$ -inseparable ontologies in this normal form through standard CNF transformation techniques and through the introduction of fresh concept names which we call *definers*. For instance, we would replace an axiom $A \sqsubseteq \exists r.(B \sqcup C)$ by two clauses $\neg A \sqcup \exists r.D$, $\neg D \sqcup B \sqcup C$, where D is an introduced definer. These definers play a central role in the method. We can normalize ontologies so that every clause contains at most one negative occurrence of a definer. This invariant ensures that for every definer D , we can transform the clauses that contain $\neg D$ into a single GCI of the form $D \sqsubseteq C$, and then eliminate D again by using Ackermann’s lemma or its generalization. The resolution procedure produces only clauses that also satisfy this invariant, which is why all definers can be eliminated in a final step.

The resolution method used by LETHE uses the following two inference rules:

$$\textbf{Resolution: } \frac{C_1 \sqcup A \quad C_2 \sqcup \neg A}{C_1 \sqcup C_2} \qquad \textbf{Role Propagation: } \frac{C_1 \sqcup \forall r.D_1 \quad C_2 \sqcup \text{Q}r.D_2}{C_1 \sqcup C_2 \sqcup \text{Q}r.D_{12}}$$

where $\text{Q} \in \{\exists, \forall\}$, and D_{12} is a possibly fresh definer that represents $D_1 \sqcap D_2$. We can introduce such a definer by adding the clauses $\neg D_{12} \sqcup D_1$, $\neg D_{12} \sqcup D_2$ and immediately

resolving on D_1 and D_2 . In order to ensure termination, the method keeps track of introduced definers and reuses them where possible—for details we refer to [76].

The idea is now to perform all possible inferences on the concept names we want to eliminate, where we avoid inferences that would introduce a clause with two negative occurrences of a definer. Because of this invariant, some resolution inferences become only possible through application of the role propagation rule, as illustrated in the following example. Once all inferences on a concept name to be eliminated have been applied, we can remove all occurrences of that concept name.² Once we have eliminated all concept names outside of the signature Σ in this way, we eliminate all introduced definers using Ackermann’s lemma and its generalization to obtain the uniform interpolant. To eliminate a role name r , we perform all possible inferences on r using role propagation. Afterwards, we can remove literals $\exists r.D$ for which D is unsatisfiable, and filter out the remaining occurrences of r .³

► **Example 13.** We want to compute the $\{A, B, D, E, r\}$ -interpolant of the following ontology:

$$\mathcal{O} = \{ \quad A \sqsubseteq \exists r.(B \sqcap C) \quad \exists r.(C \sqcap D) \sqsubseteq E \quad \}$$

The corresponding set of DL clauses is the following:

$$1. \neg A \sqcup \exists r.D_1 \quad 2. \neg D_1 \sqcup B \quad 3. \neg D_1 \sqcup C \quad 4. \forall r.D_2 \sqcup E \quad 5. \neg D_2 \sqcup \neg C \sqcup \neg D$$

We need to eliminate C . We cannot resolve on Clauses 3 and 5, since it would produce a clause with two negative occurrences of a definer, which would break our invariant. To make resolution on C possible, we need to first apply role propagation on Clauses 1 and 4, followed by an immediate resolution on D_1 and D_2 :

$$\begin{array}{lll} 6. \neg A \sqcup E \sqcup \exists r.D_{12} & 7. \neg D_{12} \sqcup D_1 & 8. \neg D_{12} \sqcup D_2 \\ 9. \neg D_{12} \sqcup B & 10. \neg D_{12} \sqcup C & 11. \neg D_{12} \sqcup \neg C \sqcup \neg D \end{array}$$

Clauses 7 and 8 are only intermediate inferences and can be discarded. We can now resolve upon C , the name to be eliminated, namely on Clauses 10 and 11:

$$12. \neg D_{12} \sqcup \neg D$$

No further resolution steps on C are possible without breaking the invariant that every clause contains at most one negative occurrence of a definer. The procedure would now remove all clauses containing C (Clause 3, 5, 10 and 11) and apply Ackermann’s lemma to eliminate the

² This is essentially the same idea as in the resolution-based approach for propositional logic discussed in Chapter 1, or in the second-order quantifier elimination tool SCAN [105] described in Chapter 12.

³ [79] discusses a more involved method for role elimination. Both methods are implemented in the current version of LETHE [77, 76].

18 Interpolation in Knowledge Representation

introduced names D_1 , D_2 and D_{12} again. The resulting set of DL clauses can be represented as the following set of CIs, which is the desired uniform interpolant:

$$\{ \quad A \sqsubseteq \exists r.B, \quad A \sqcap \forall r.(\neg B \sqcup D) \sqsubseteq E \quad \}$$

Note that while this approach will always compute a uniform interpolant in $\mathcal{ALC}\mu$, it may not find a uniform interpolant without fixpoints even if one exists. As argued above, in practice, one can keep the problematic definers, or compute approximations of the fixpoint expressions. A practical method that is able to decide the existence of uniform interpolants without fixpoints on realistic ontologies has not been found so far.

3.3 Related Notions and Applications

We close this section by discussing four relevant notions closely related to uniform interpolation: inseparability and conservative extensions, logical difference, and modules.

Inseparability and Conservative Extensions. Inseparability as in Definition 4 is a key property that is desired in many applications. For instance, when an ontology evolves by adding, removing, or changing concept inclusions, we might want to ensure that the change does not affect entailments in a certain signature of interest. Since inseparability is built-in into uniform interpolation, there are natural connections to these applications; we discuss some of them.

Very closely related to uniform interpolation is the notion of conservative extensions. Conservative extensions are a classical notion studied in logic which has been introduced to KR in the seminal work [47]. Let $\mathcal{L}$ be a DL and $\mathcal{O}_1$, $\mathcal{O}_2$ be two $\mathcal{L}$ ontologies. Recall that we write $\mathcal{O}_1 \equiv_{\Sigma}^{\mathcal{L}} \mathcal{O}_2$ to indicate that $\mathcal{O}_1$ and $\mathcal{O}_2$ entail precisely the same concept inclusions that can be expressed in $\mathcal{L}$ using signature Σ . We call $\mathcal{O}_2$ a *conservative extension* of $\mathcal{O}_1$ if $\mathcal{O}_1 \subseteq \mathcal{O}_2$ and $\mathcal{O}_1 \equiv_{\text{sig}(\mathcal{O}_1)}^{\mathcal{L}} \mathcal{O}_2$. Conservative extensions are relevant when we want to extend an ontology with new concept inclusions about new symbols, while preserving the behaviour with respect to the original signature. Hence, the central problem is to decide given $\mathcal{O}_1, \mathcal{O}_2$ whether $\mathcal{O}_2$ is a conservative extension of $\mathcal{O}_1$. As pointed out in [47], it is immediate from the definitions that

$$\mathcal{O}_2 \text{ is a conservative extension of } \mathcal{O}_1 \text{ iff } \mathcal{O}_1 \text{ is a uniform } \mathcal{L}(\text{sig}(\mathcal{O}_1))\text{-interpolant of } \mathcal{O}_2. \quad (13)$$

This can in some cases be exploited to decide conservative extensions. Indeed, if $\mathcal{O}_2$ has a uniform $\mathcal{L}(\text{sig}(\mathcal{O}_1))$ -interpolant, then we can compute it and check equivalence with $\mathcal{O}_1$. As discussed before, this is however not always possible since uniform interpolants may not exist. In general, even when not only adding concept inclusions, we may want to decide whether two ontologies are inseparable. As for conservative extensions, uniform interpolants could help also in this case. Indeed, if $\mathcal{O}_1^{\Sigma}$ and $\mathcal{O}_2^{\Sigma}$ are uniform $\mathcal{L}(\Sigma)$ -interpolants of $\mathcal{O}_1$ and $\mathcal{O}_2$, respectively, then $\mathcal{O}_1 \equiv_{\Sigma}^{\mathcal{L}} \mathcal{O}_2$ iff $\mathcal{O}_1^{\Sigma} \equiv \mathcal{O}_2^{\Sigma}$, that is, $\mathcal{O}_1^{\Sigma} \models \mathcal{O}_2^{\Sigma}$ and $\mathcal{O}_2^{\Sigma} \models \mathcal{O}_1^{\Sigma}$.

Ontology Language	Complexity
$\mathcal{EL}$	EXPTIME [95]
$\mathcal{ALC}$, $\mathcal{ALCI}$	2EXPTIME [47, 94]
$\mathcal{ALCO}$	3EXPTIME-hard [65]
$\mathcal{ALCFIO}$	undecidable [94]

■ **Table 3** The complexity of uniform interpolant recognition.

Both conservative extensions and inseparability have been heavily studied for description logics, see for example [47, 94, 20, 64, 95, 65]. The complexity results obtained there have implications also on the side of uniform interpolation, especially for the recognition problem. Here, *uniform interpolant recognition* is the problem of deciding whether an ontology $\mathcal{O}^\Sigma$ is a uniform $\mathcal{L}(\Sigma)$ -interpolant of another ontology $\mathcal{O}$. The recognition problem has not been explicitly studied in the literature, but it might be relevant for cases when the ontology engineer has a candidate ontology in mind and wants to verify whether it is indeed a uniform interpolant. Table 3 summarizes some complexity results for the recognition problem. The lower bounds are inherited from deciding conservative extensions via the reduction (13). For the upper bounds, observe that uniform interpolant recognition reduces to entailment and inseparability as follows:

$$\mathcal{O}^\Sigma \text{ is a uniform } \mathcal{L}(\Sigma)\text{-interpolant of } \mathcal{O} \text{ iff } \mathcal{O} \models \mathcal{O}^\Sigma \text{ and } \mathcal{O} \equiv_{\Sigma}^{\mathcal{L}} \mathcal{O}^\Sigma.$$

It is worth noting that nominals are particularly challenging in this context, and we will see in Section 4 that this is also the case for interpolation at the level of concepts.

We point out that in applications related to query answering, we need alternative notions of inseparability (and thus uniform interpolation), namely inseparability *by queries*. As this is beyond the scope of this chapter, we refer the reader to [20] for a survey.

Logical Difference. In case two ontologies are not $\mathcal{L}(\Sigma)$ -inseparable, a natural question is in which $\mathcal{L}(\Sigma)$ -entailments they actually differ, which is captured by the logical difference of the ontologies:

► **Definition 14** (Logical Difference). *Let $\mathcal{O}_1, \mathcal{O}_2$ be two ontologies and Σ be a signature. The logical difference from $\mathcal{O}_1$ to $\mathcal{O}_2$ in Σ is defined as*

$$\text{diff}(\mathcal{O}_1, \mathcal{O}_2, \Sigma) = \{\alpha \mid \text{sig}(\alpha) \subseteq \Sigma, \mathcal{O}_1 \models \alpha, \mathcal{O}_2 \not\models \alpha\}$$

Logical difference is particularly useful to track changes between different versions of an ontology. In contrast to a purely syntactic check (which CIs have been added/removed/changed), logical difference provides a semantic way to understand whether entailments in a signature of interest have changed. Note that if the logical difference between two ontologies is non-empty, then it contains infinitely many concept inclusions. Nonetheless, it is often possible to compute

20 Interpolation in Knowledge Representation

finite representations [72]. For more expressive logics, the only existing tools compute finite representations of the logical difference by reduction to uniform interpolation [89, 76, 88].

Modules. As discussed in the introduction, one key application of uniform interpolation is ontology reuse. Modules are an alternative solution for this. Modules work the other way around to conservative extensions: given an ontology $\mathcal{O}$ and a signature Σ , a $\mathcal{L}(\Sigma)$ -module of $\mathcal{O}$ is an $\mathcal{L}(\Sigma)$ -inseparable subset of $\mathcal{O}$. Modules are investigated in detail in [55, 70, 71], and are relevant for the application *Modularisation and Reuse* discussed in the introduction, but they have also been used for other purposes such as to improve reasoner performance [109], or as preprocessing step for uniform interpolant computation [89, 76, 131]. Differently to uniform interpolants, the size of a module is bounded by the size of the original ontology, and the syntactical structure of CIs is not changed, which is sometimes relevant. At the same time, uniform interpolants can often be more compact than modules, and give guarantees on the used signature. Practical methods for uniform interpolation have been used to compute subset-minimal $\mathcal{EL}(\Sigma)$ and $\mathcal{ALC}(\Sigma)$ -modules [77, 130], but there are also very fast syntactical methods that compute $\text{SOL}(\Sigma)$ -modules [55], which may however not be subset-minimal. A compromise between modules and uniform interpolants that can leverage the advantages of both are *generalised modules*. A generalised $\mathcal{L}(\Sigma)$ -module of an ontology $\mathcal{O}$ can be just any ontology that is $\mathcal{L}(\Sigma)$ -inseparable with $\mathcal{O}$, but is ideally smaller and simpler than $\mathcal{O}$. General modules can be significantly simpler than both uniform interpolants and modules [102, 129]. The method in [129] uses a technique similar to that of LETHE discussed in this chapter.

4 Craig Interpolation for Description Logic Concepts

In this section, we will define and discuss interpolation in description logics at the level of concepts rather than at the level of ontologies as in preceding section. We start in Section 4.1 with (Craig) interpolation and then move in Section 4.2 to the tightly related concept of Beth definability. In Section 4.3, we discuss the main applications of interpolants and explicit definitions which motivate the necessity of computing them. The computation problem is then covered in Section 4.4.

4.1 Craig Interpolation

We use the following standard definition of interpolants in the context of DL ontologies.

► **Definition 15 (Interpolants).** *Let $\mathcal{L}$ be any DL. Let C_1, C_2 be $\mathcal{L}$ concepts, $\mathcal{O}$ an $\mathcal{L}$ ontology and Σ be a signature. Then, an $\mathcal{L}(\Sigma)$ -interpolant for C_1 and C_2 under $\mathcal{O}$ is any $\mathcal{L}(\Sigma)$ concept I that satisfies $\mathcal{O} \models C_1 \sqsubseteq I$ and $\mathcal{O} \models I \sqsubseteq C_2$.*

We remark that this definition of interpolants is in line with the one used for modal logic in Chapter 4. More precisely, with an empty ontology $\mathcal{O}$, an interpolant for C_1 and C_2 is an interpolant for the validity $\models C_1 \rightarrow C_2$, when viewing C_1, C_2 as modal logic formulas.

The case with ontologies is also related to the setting of interpolation *relative to theories* in first-order logic, see Chapter 2.

A classical application of interpolants in DLs is in explaining subsumption relations. More precisely, the idea put forward in [115] is that an $\mathcal{L}(\Sigma)$ -interpolant for C and D with $\models C \sqsubseteq D$ for a *minimal* Σ is a good explanation of the possibly complicated subsumption $C \sqsubseteq D$. The following example illustrates the idea.

► **Example 16.** Consider $C = \exists \text{child}.\top \sqcap \forall \text{child}.\text{Doctor}$ and $D = \exists \text{child}.\text{Doctor} \sqcup \text{Rich}$. Clearly, $\models C \sqsubseteq D$ and $\exists \text{child}.\text{Doctor}$ is an $\mathcal{ALC}(\{\text{child}, \text{Doctor}\})$ -interpolant of C and D . Moreover, one can show that there is no $\mathcal{ALC}(\Sigma)$ -interpolant of C and D for $\Sigma \subsetneq \{\text{child}, \text{Doctor}\}$. Hence, $\exists \text{child}.\text{Doctor}$ is an arguably simple explanation of the subsumption. $\dashv$

This application motivates the study of the following $\mathcal{L}$ -interpolant existence problem:

Input $\mathcal{L}$ ontology $\mathcal{O}$, $\mathcal{L}$ concepts C_1, C_2 and a signature Σ .

Question Does there exist an $\mathcal{L}(\Sigma)$ -interpolant for C_1 and C_2 under $\mathcal{O}$?

We will see that this potentially difficult existence problem has a surprisingly simple solution for DLs that satisfy the *Craig interpolation property* (CIP), defined following [119, 8]. Recall that $\text{sig}(X)$ denotes the set of concept and role names used in object X ; we use $\text{sig}(\mathcal{O}, C)$ to abbreviate $\text{sig}(\mathcal{O}) \cup \text{sig}(C)$.

► **Definition 17** (Craig Interpolation Property). *Let $\mathcal{L}$ be any DL. We say that $\mathcal{L}$ enjoys the Craig interpolation property (CIP) if for every $\mathcal{L}$ concepts C_1, C_2 and $\mathcal{L}$ ontologies $\mathcal{O}_1, \mathcal{O}_2$ with $\mathcal{O}_1 \cup \mathcal{O}_2 \models C_1 \sqsubseteq C_2$, there exists a Craig interpolant, that is, an $\mathcal{L}(\Sigma)$ -interpolant of C_1 and C_2 under $\mathcal{O}_1 \cup \mathcal{O}_2$ where $\Sigma = \text{sig}(\mathcal{O}_1, C_1) \cap \text{sig}(\mathcal{O}_2, C_2)$.*

In the same way as the definition of interpolants, the definition of the CIP under empty ontologies $\mathcal{O}_1 = \mathcal{O}_2 = \emptyset$ coincides with the standard definition of the CIP in modal logic. The split of the ontology into two parts is necessary to ensure the existence of Craig interpolants in the presence of ontologies. Indeed, consider the following example from [8]. Let $\mathcal{O} = \{A_1 \sqsubseteq A_2, A_2 \sqsubseteq A_3\}$, and consider concepts $C_1 = A_1$ and $C_2 = A_3$. Then for every $\mathcal{O}_1, \mathcal{O}_2$ with $\mathcal{O} = \mathcal{O}_1 \cup \mathcal{O}_2$, there is an $\mathcal{ALC}(\Sigma)$ -interpolant for C_1 and C_2 under $\mathcal{O}$ with $\Sigma = \text{sig}(\mathcal{O}_1, C_1) \cap \text{sig}(\mathcal{O}_2, C_2)$. However, there is no $\mathcal{ALC}(\Sigma_0)$ -interpolant for C_1 and C_2 under $\mathcal{O}$ with $\Sigma_0 = \text{sig}(C_1) \cap \text{sig}(C_2) = \emptyset$.

The Craig interpolation property is so powerful since it *guarantees* the existence of interpolants/explanations in terms of the common signature $\Sigma = \text{sig}(\mathcal{O}_1, C_1) \cap \text{sig}(\mathcal{O}_2, C_2)$. Fortunately, many DLs enjoy the CIP, also in the presence of ontologies [119].

► **Theorem 18.** *$\mathcal{ALC}$ and any extension with $\mathcal{S}, \mathcal{I}, \mathcal{F}$ enjoys the Craig interpolation property.*

The proof of Theorem 18 in [119] is based on a tableau algorithm for subsumption and is constructive in the sense that it also computes a Craig interpolant in case the subsumption holds. We will next provide a shorter (though not constructive) model-theoretic proof, which

22 Interpolation in Knowledge Representation

bears a lot of similarity with the model-theoretic proof for the CIP in modal logic in Chapter 4. The main difference is that we have to accomodate the presence of an ontology. The first step of this proof is to provide a model-theoretic characterization for interpolant existence in terms of bisimulations, inspired by early works by Robinson [108]. This is a rather uniform step that can be adapted to virtually every (description) logic by “plugging in” the respective bisimulation notion capturing the expressive power of the logic. The second step is then an amalgamation lemma which shows that the model-theoretic condition is satisfied whenever the subsumption holds. This second step does not work for all logics, and we will see that failure of this step for some logic usually means that this logic does not enjoy the CIP.

We introduce the necessary notation. Let $\mathcal{L}$ be some description logic. Let C_1, C_2 be $\mathcal{L}$ concepts, $\mathcal{O}$ be an $\mathcal{L}$ ontology, and Σ be a signature. Then C_1, C_2 are called *jointly $\sim_{\mathcal{L}, \Sigma}$ -consistent under $\mathcal{O}$* if there exist models $\mathcal{I}_1, \mathcal{I}_2$ of $\mathcal{O}$ and elements $d_i \in C_i^{\mathcal{I}_i}$ for $i = 1, 2$, satisfying $\mathcal{I}_1, d_1 \sim_{\mathcal{L}, \Sigma} \mathcal{I}_2, d_2$.⁴ Joint consistency can be thought of as a strong form of satisfiability of two concepts with an additional bisimilarity requirement. The following lemma characterizes the existence of interpolants in terms of joint consistency. The proof is rather standard and relies on the fact that $\mathcal{L}$ -bisimulations capture the expressive power of $\mathcal{L}$ and crucially also on compactness, see Chapter 4 for a similar lemma, but also [53].

► **Lemma 19.** *Let C_1, C_2 be $\mathcal{L}$ concepts, $\mathcal{O}$ an $\mathcal{L}$ ontology, and Σ be a signature. Then the following conditions are equivalent:*

1. *there is no $\mathcal{L}(\Sigma)$ -interpolant for C_1 and C_2 under $\mathcal{O}$;*
2. *C_1 and $\neg C_2$ are jointly $\sim_{\mathcal{L}, \Sigma}$ -consistent under $\mathcal{O}$.*

Intuitively, for an $\mathcal{L}(\Sigma)$ -interpolant for C_1 and C_2 to exist, the inconsistency of C_1 and $\neg C_2$ must be detectable by an $\mathcal{L}(\Sigma)$ -bisimulation, since otherwise we cannot express it using an $\mathcal{L}(\Sigma)$ concept. Hence, in order to show the CIP for a DL, it suffices to show that a witness for Point 2 can be turned into a witness for failure of the subsumption $\mathcal{O} \models C_1 \sqsubseteq C_2$. This is the content of the following amalgamation lemma, which is stated and proved for $\mathcal{ALC}$ only, for the sake of simplicity. Its proof is similar to the proof of an analogous amalgamation lemma for modal logic in Chapter 4. We refer the reader to Chapter 8 for more on amalgamation.

► **Lemma 20.** *Let $\mathcal{I}_1, \mathcal{I}_2$ be models of an $\mathcal{ALC}$ ontology $\mathcal{O}_1 \cup \mathcal{O}_2$ and suppose $\mathcal{I}_1, d_1 \sim_{\mathcal{ALC}, \Sigma_1 \cap \Sigma_2} \mathcal{I}_2, d_2$ for $d_1 \in \Delta^{\mathcal{I}_1}, d_2 \in \Delta^{\mathcal{I}_2}$ and signatures $\Sigma_1 \supseteq \text{sig}(\mathcal{O}_1), \Sigma_2 \supseteq \text{sig}(\mathcal{O}_2)$. Then there is a model $\mathcal{J}$ of $\mathcal{O}_1 \cup \mathcal{O}_2$ and $e \in \Delta^{\mathcal{J}}$ such that $\mathcal{J}, e \sim_{\mathcal{ALC}, \Sigma_1} \mathcal{I}_1, d_1$ and $\mathcal{J}, e \sim_{\mathcal{ALC}, \Sigma_2} \mathcal{I}_2, d_2$.*

As announced, Lemmas 19 and 20 can be used to prove that $\mathcal{ALC}$ enjoys the CIP.

⁴ It is worth noting that *joint $\sim_{\mathcal{L}, \Sigma}$ -consistency* is dual to the notion of *entailment along bisimulations* in Chapter 4. Indeed, C_1 and $\neg C_2$ are jointly $\sim_{\mathcal{L}, \Sigma}$ -consistent iff C_1 does not entail C_2 along $\mathcal{L}(\Sigma)$ -bisimulations. We stick to joint bisimilarity since it has been the term recently used in the DL community.

Proof of Theorem 18 for $\mathcal{ALC}$. Suppose $\mathcal{O}_1 \cup \mathcal{O}_2 \models C_1 \sqsubseteq C_2$ for $\mathcal{ALC}$ ontologies $\mathcal{O}_1, \mathcal{O}_2$ and $\mathcal{ALC}$ concepts C_1, C_2 with $\Sigma_1 = \text{sig}(\mathcal{O}_1, C_1)$ and $\Sigma_2 = \text{sig}(\mathcal{O}_2, C_2)$. If there is no Craig interpolant for C_1 and C_2 under $\mathcal{O}_1 \cup \mathcal{O}_2$, then by Lemma 19, there are models $\mathcal{I}_1, \mathcal{I}_2$ of $\mathcal{O}_1 \cup \mathcal{O}_2$ and $d_1 \in \Delta^{\mathcal{I}_1}, d_2 \in \Delta^{\mathcal{I}_2}$ such that $d_1 \in C_1^{\mathcal{I}_1}, d_2 \notin C_2^{\mathcal{I}_2}$, and $\mathcal{I}_1, d_1 \sim_{\mathcal{ALC}, \Sigma_1 \cap \Sigma_2} \mathcal{I}_2, d_2$. By Lemma 20, there is a model $\mathcal{J}$ of $\mathcal{O}_1 \cup \mathcal{O}_2$ and $e \in \Delta^{\mathcal{J}}$ such that $\mathcal{J}, e \sim_{\mathcal{ALC}, \Sigma_1} \mathcal{I}_1, d_1$ and $\mathcal{J}, e \sim_{\mathcal{ALC}, \Sigma_2} \mathcal{I}_2, d_2$. Now, since $d_1 \in C_1^{\mathcal{I}_1}$, $\text{sig}(C_1) \subseteq \Sigma_1$, and $\mathcal{J}, e \sim_{\mathcal{ALC}, \Sigma_1} \mathcal{I}_1, d_1$, we also have $e \in C_1^{\mathcal{J}}$. Analogously, one can show that $d_2 \notin C_2^{\mathcal{J}}$. This is in contradiction to the assumed subsumption $\mathcal{O}_1 \cup \mathcal{O}_2 \models C_1 \sqsubseteq C_2$. ◀

We come back to the problem of $\mathcal{L}$ -interpolant existence. As mentioned above, the problem trivializes for DLs that enjoy the CIP if we ask for interpolants in the common signature. Moreover, it is not difficult to reduce the existence of $\mathcal{L}(\Sigma)$ -interpolants for given signature Σ to the existence of Craig interpolants. Indeed, one can verify that, for any standard DL $\mathcal{L}$, the following are equivalent for all $\mathcal{L}$ concepts C_1, C_2 , $\mathcal{L}$ ontologies $\mathcal{O}$, signatures Σ , and $C_{2\Sigma}$ and $\mathcal{O}_\Sigma$ obtained from C_2 and $\mathcal{O}$, respectively, by renaming all symbols not in Σ uniformly to fresh symbols:

- there is an $\mathcal{L}(\Sigma)$ -interpolant for C_1 and C_2 under $\mathcal{O}$;
- there is a Craig interpolant for C_1 and $C_{2\Sigma}$ under $\mathcal{O} \cup \mathcal{O}_\Sigma$.

Since existence of Craig interpolants coincides with the respective subsumption relationship in DLs enjoying the CIP, we have reduced $\mathcal{L}$ -interpolant existence to subsumption checking for such DLs. A reduction of subsumption to interpolant existence is also possible. Since subsumption in all DLs mentioned in Theorem 18 is EXPTIME-complete [120], we obtain:

► **Theorem 21.** *For any DL $\mathcal{L}$ in Theorem 18, $\mathcal{L}$ -interpolant existence is EXPTIME-complete.*

It is worth noting that the proof of Theorem 18 (and thus of Theorem 21) is not constructive since the proof of the model-theoretic characterization in Lemma 19 relies on compactness and does not provide an interpolant if one exists. We will address the computation problem in Section 4.4.

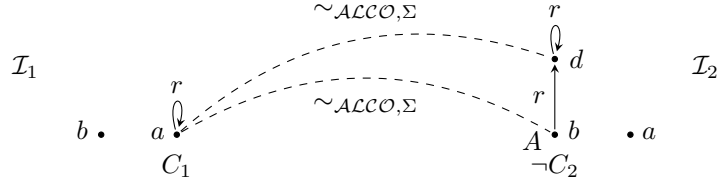
Here, we will turn our attention to the fact that, unfortunately, not all DLs enjoy the CIP.

► **Theorem 22** ([119, 8]). *$\mathcal{ALCCO}$ and $\mathcal{ALCH}$ and their extensions by $\mathcal{S}, \mathcal{I}, \mathcal{F}$ do not enjoy the Craig interpolation property.*

Proof. We show the argument only for $\mathcal{ALCCO}$, for $\mathcal{ALCH}$ and their extensions see [119, 8]. Consider $\mathcal{ALCCO}$ concepts $C_1 = \{a\} \sqcap \exists r.\{a\}$ and $C_2 = A \rightarrow \exists r.A$ with common signature $\{r\}$. Then $\models C_1 \sqsubseteq C_2$, but there is no $\mathcal{ALCCO}(\{r\})$ -interpolant for C_1 and C_2 . Indeed, the interpretations in Figure 1 witness that C_1 and $\neg C_2$ are jointly $\sim_{\mathcal{ALCCO}, \{r\}}$ -consistent, and by Lemma 19 there is no Craig interpolant. ◀

One consequence of the lack of the CIP is that the reduction of interpolant existence to subsumption checking used to prove Theorem 21 does not work anymore. Fortunately, it

24 Interpolation in Knowledge Representation



■ **Figure 1** Interpretations $\mathcal{I}_1$ and $\mathcal{I}_2$ illustrating the proof of Theorem 22.

turns out that the problem is still decidable, although typically harder than subsumption. We exploit that Lemma 19 provides a reduction of interpolant existence to the problem of deciding joint consistency under ontologies, so it suffices to decide the latter. For didactic purposes and since we need it later, we provide first a relatively simple algorithm for deciding joint consistency in $\mathcal{ALC}$, which is inspired by standard type elimination algorithms for deciding satisfiability in DLs [11] and is closely related to the type elimination sequences in Chapter 4.

To describe the algorithm deciding joint consistency, let us fix an $\mathcal{ALC}$ ontology $\mathcal{O}$, $\mathcal{ALC}$ concepts C_1, C_2 , and a signature Σ . Let Γ denote the set of all subconcepts that occur in $\mathcal{O}$, C_1, C_2 , closed under single negation. A *type* is any set $t \subseteq \Gamma$ such that there is a model $\mathcal{I}$ of $\mathcal{O}$ and an element $d \in \Delta^{\mathcal{I}}$ such that $t = \text{tp}_{\mathcal{I}}(d)$ where, as in Section 3.1,

$$\text{tp}_{\mathcal{I}}(d) = \{C \in \Gamma \mid d \in C^{\mathcal{I}}\}.$$

A *mosaic* m is a pair $m = (t_1, t_2)$ of types. Intuitively, a mosaic (t_1, t_2) provides an abstract description of two elements d_1, d_2 in two models $\mathcal{I}_1, \mathcal{I}_2$ of $\mathcal{O}$ which have types t_1, t_2 and are $\mathcal{ALC}(\Sigma)$ -bisimilar. Of course, not all mosaics are realizable in this sense and the goal of the elimination is to identify those that are.

We write $t \rightsquigarrow_r t'$ if an element of type t' is a viable r -successor of an element of type t , that is, $\{C \mid \forall r.C \in t\} \subseteq t'$. We use $(t_1, t_2) \rightsquigarrow_r (t'_1, t'_2)$ to abbreviate $t_1 \rightsquigarrow_r t'_1, t_2 \rightsquigarrow_r t'_2$. Let $\mathcal{M}$ be a set of mosaics. We call $(t_1, t_2) \in \mathcal{M}$ *bad* if it violates one of the following conditions:

(Atomic Consistency) for every $A \in \Sigma$, $A \in t_1$ iff $A \in t_2$;

(Existential Saturation) for every $r \in \Sigma$, every $i = 1, 2$, and every $\exists r.C \in t_i$, there is $(t'_1, t'_2) \in \mathcal{M}$ such that $C \in t'_i$ and $(t_1, t_2) \rightsquigarrow_r (t'_1, t'_2)$.

Clearly, a mosaic (t_1, t_2) violating atomic consistency cannot be realized as required, due to concept name $A \in \Sigma$. Similarly, a mosaic violating existential saturation cannot be realized since it lacks a viable r -successor for one of the t_i . We have the following characterization of joint consistency:

► **Lemma 23.** C_1 and C_2 are jointly $\sim_{\mathcal{ALC}, \Sigma}$ -consistent under $\mathcal{O}$ iff there is a set $\mathcal{M}^*$ of mosaics that does not contain bad mosaics and some $(t_1, t_2) \in \mathcal{M}^*$ with $C_1 \in t_1$ and $C_2 \in t_2$.

The existence of an $\mathcal{M}^*$ as in Lemma 23 can be decided by a simple mosaic elimination algorithm. The idea is to compute a sequence of mosaics

$$\mathcal{M}_0, \mathcal{M}_1, \mathcal{M}_2, \dots$$

where $\mathcal{M}_0$ is the set of all mosaics, and for $i \geq 0$, $\mathcal{M}_{i+1}$ is obtained from $\mathcal{M}_i$ by eliminating all bad mosaics from $\mathcal{M}_i$. Since $\mathcal{M}_0$ is finite, this process reaches a fixpoint after finitely many steps. Then, C_1 and C_2 are jointly $\sim_{\mathcal{ALC}, \Sigma}$ -consistent under $\mathcal{O}$ iff the fixpoint $\mathcal{M}^*$ of that process contains (t_1, t_2) as in Lemma 23. The algorithm runs in exponential time in the size of the input, since the number of mosaics is at most exponential in the size of the input, and each mosaic can be checked for badness in time polynomial in the number of mosaics. Thus, the approach to interpolant existence via deciding joint consistency is not worse than the sketched reduction to subsumption checking. We will see in Section 4.4 how to extend it to compute interpolants as well.

This idea can be generalized to decide $\mathcal{L}$ -interpolant existence for DLs lacking the CIP.

► **Theorem 24** ([8]). *For $\mathcal{L} \in \{\mathcal{ALCCO}, \mathcal{ALCH}, \mathcal{ALCOT}, \mathcal{ALCHT}\}$, $\mathcal{L}$ -interpolant existence is 2EXPTIME-complete.*

Due to Lemma 19, it suffices to show the results for joint consistency. We sketch it here for $\mathcal{ALCCO}$, see [8] for a full proof and the other DLs covered in the theorem. To get an intuition for the lower bound proof, it is instructive to look again at the proof of Theorem 22 and particularly Figure 1, where the elements b, d in $\mathcal{I}_2$ are *forced* to be bisimilar, as they are both bisimilar to a in $\mathcal{I}_1$. This idea is extended to force *exponentially many* elements to be bisimilar in witnesses for joint $\sim_{\mathcal{L}, \Sigma}$ -consistency, which in turn is exploited to synchronize configurations of exponentially space bounded alternating Turing machines.

For the upper bound we extend the mosaic elimination algorithm that was used to decide joint consistency for $\mathcal{ALC}$ to $\mathcal{ALCCO}$. Let $\mathcal{O}$ be an $\mathcal{ALCCO}$ ontology, C_1, C_2 be $\mathcal{ALCCO}$ concepts, and Σ be a signature. Let Γ be again the set of all subconcepts of $\mathcal{O}, C_1, C_2$. A *type* is defined as before as a subset of Γ that is realizable in a model of $\mathcal{O}$. To address that one can force elements to be bisimilar, we have to extend the notion of a mosaic m to be a pair (T_1, T_2) of *sets* of types. Intuitively, a mosaic (T_1, T_2) describes collections of elements in two interpretations $\mathcal{I}_1, \mathcal{I}_2$ which realize precisely the types in T_1, T_2 and are mutually $\mathcal{ALCCO}(\Sigma)$ -bisimilar. Naturally, not all such mosaics can be realized as described and the goal is to find the realizable ones. We write $(T_1, T_2) \rightsquigarrow_r (T'_1, T'_2)$ if for $i = 1, 2$ and every $t \in T_i$, there is $t' \in T'_i$ with $t \rightsquigarrow_r t'$, that is, every type in T_i has a viable r -successor in T'_i .

Let $\mathcal{M}$ be a set of mosaics. A mosaic $(T_1, T_2) \in \mathcal{M}$ is *bad* if it violates one of the conditions atomic consistency and existential saturation, suitably generalized to sets of types:

(Atomic Consistency) for every $A \in \Sigma$ and every $t, t' \in T_1 \cup T_2$, $A \in t$ iff $A \in t'$;

(Existential Saturation) for every $r \in \Sigma$, $i = 1, 2$, every $t \in T_i$, and every $\exists r.C \in t$, there is $(T'_1, T'_2) \in \mathcal{M}$ with $(T_1, T_2) \rightsquigarrow_r (T'_1, T'_2)$ and such that $C \in t'$ for some $t' \in T'_i$ with $t \rightsquigarrow_r t'$.

26 Interpolation in Knowledge Representation

We need a further property of sets of mosaics to ensure that nominals are handled correctly: they are realized precisely once in every interpretation. A set $\mathcal{M}$ of mosaics is *good for nominals* if for every individual $a \in \Gamma$ and $i = 1, 2$, there is exactly one t_a^i with $\{a\} \in t_a^i \in \bigcup_{(T_1, T_2) \in \mathcal{M}} T_i$ and exactly one pair $(T_1, T_2) \in \mathcal{M}$ with $t_a^i \in T_i$. Moreover, if $a \in \Sigma$, then the pair takes one of the forms $(\{t_a^1\}, \{t_a^2\})$, $(\emptyset, \{t_a^2\})$, or $(\{t_a^1\}, \emptyset)$. One can then show:

► **Lemma 25** (Lemma 6.5 in [8]). *C_1 and C_2 are jointly $\sim_{\mathcal{ALCO}, \Sigma}$ -consistent iff there is a set $\mathcal{M}^*$ of mosaics that is good for nominals and contains no bad mosaic such that there is $(T_1, T_2) \in \mathcal{M}^*$ and $t_1 \in T_1, t_2 \in T_2$ with $C_1 \in t_1$ and $C_2 \in t_2$.*

The upper bound in Theorem 24 now follows from the fact that there are only double exponentially many mosaics and that an $\mathcal{M}^*$ as in Lemma 25 can be found (if it exists) by eliminating mosaics from all *maximal* sets of mosaics that are good for nominals. The upper bound for $\mathcal{ALCH}$ and the other logics in Theorem 24 is similar. In fact, mosaic elimination has been used to show decidability of interpolant existence also for other logics that related to DLs and relevant to KR [67, 81, 82].

$\mathcal{EL}$ and its Relatives. Given the importance of the lightweight DL $\mathcal{EL}$ in applications, we report also on the (surprisingly different) situation in $\mathcal{EL}$ and its relatives. While $\mathcal{EL}$ itself and its extension with role hierarchies or transitive roles do enjoy the CIP [93, 33], most other extensions of $\mathcal{EL}$ such as with inverse roles, nominals, or the combination of transitive roles and role hierarchies do not [33]. Thus, the extension with $\mathcal{H}$ does not lead to the loss of the CIP in $\mathcal{EL}$, but it does in $\mathcal{ALC}$, and conversely, the extension with $\mathcal{I}$ does lead to the the loss of the CIP in $\mathcal{EL}$, but not in $\mathcal{ALC}$. As a rule of thumb, the complexity of the interpolant existence problem in an extension of $\mathcal{EL}$ without CIP coincides with the complexity of subsumption in that extension [33]. This is in contrast to $\mathcal{ALC}$ where interpolant existence is generally one exponent more difficult than subsumption for extensions without CIP.

Repairing the CIP. We conclude this part with a discussion of *repairing* the lack of the CIP by allowing interpolants from a richer logic. A notable positive instance of this approach is to allow formulas in the guarded, two-variable fragment (GF^2) of FO as interpolants under $\mathcal{ALCH}$ ontologies: GF^2 extends $\mathcal{ALCH}$ and it is known that GF^2 does enjoy the CIP [59, 60]. Consequently, there is a GF^2 -interpolant for every valid subsumption in $\mathcal{ALCH}$. The situation is more complicated in the case of $\mathcal{ALCO}$. While it would be interesting to investigate existence of interpolants in logics more expressive than $\mathcal{ALCO}$, say GF^2 with constants, it has been shown that (under mild conditions) there is no *decidable* extension of $\mathcal{ALCO}$ that enjoys CIP [117]. Hence, there is no decidable logic which guarantees the existence of interpolants for every valid subsumption in $\mathcal{ALCO}$.

4.2 Beth Definability

Since interpolation is intimately tied to definability, c.f. Chapter 2 and Chapter 14, and since definability is a central topic when working with DL ontologies, in this section we take a closer

look at this connection in the context of DLs. We start with introducing the relevant notions. Recall that, for a signature Σ , the Σ -reduct $\mathcal{I}|_\Sigma$ of an interpretation $\mathcal{I}$ is the interpretation obtained from $\mathcal{I}$ by dropping the interpretation of all symbols not in Σ .

► **Definition 26** (Explicit/Implicit Definability). *Let $\mathcal{L}$ be any DL. Let $\mathcal{O}$ be an $\mathcal{L}$ ontology, C, C_0 be $\mathcal{L}$ concepts, and Σ be a signature. We say that C_0 is:*

- explicitly $\mathcal{L}(\Sigma)$ -definable under $\mathcal{O}$ and C if there is an explicit $\mathcal{L}(\Sigma)$ -definition of C_0 under $\mathcal{O}$ and C , that is, an $\mathcal{L}(\Sigma)$ concept D satisfying $\mathcal{O} \models C \sqsubseteq (C_0 \leftrightarrow D)$, and
- implicitly Σ -definable under $\mathcal{O}$ and C if for all models $\mathcal{I}$ and $\mathcal{J}$ of $\mathcal{O}$ with $\mathcal{I}|_\Sigma = \mathcal{J}|_\Sigma$ and all $d \in C^\mathcal{I}$, we have $d \in C_0^\mathcal{I}$ iff $d \in C_0^\mathcal{J}$.

Intuitively, C_0 is implicitly Σ -definable under $\mathcal{O}$ and C if for every model $\mathcal{I}$ of $\mathcal{O}$ and all $d \in C^\mathcal{I}$, the Σ -reduct $\mathcal{I}|_\Sigma$ “determines” whether $d \in C_0^\mathcal{I}$. Implicit definability can be equivalently defined in terms of subsumption: C_0 is implicitly Σ -definable under $\mathcal{O}$ and C iff

$$\mathcal{O} \cup \mathcal{O}_\Sigma \models C \sqcap C_0 \sqsubseteq C_\Sigma \rightarrow C_{0\Sigma} \quad (14)$$

where $\mathcal{O}_\Sigma$, C_Σ , and $C_{0\Sigma}$ are obtained from $\mathcal{O}$, C and C_0 , respectively, by replacing every non- Σ symbol uniformly by a fresh symbol. It is worth noting that a common alternative definition of explicit/implicit definability does not use the context concept C ; it is a special case of the above definition in which C is set to $\top$. We use the more general definition in order to establish a stronger relation to Craig interpolation later on. The following example illustrates implicit and explicit definability and makes do with the simpler definition.

► **Example 27.** Consider the $\mathcal{ALC}$ ontology consisting of the following CIs:

$$\begin{array}{lll} \text{Parent} \equiv \exists \text{child}.\top & \text{Parent} \equiv \text{Father} \sqcup \text{Mother} & \\ \text{Father} \sqsubseteq \text{Man} & \text{Mother} \sqsubseteq \text{Woman} & \text{Man} \sqsubseteq \neg \text{Woman} \end{array}$$

Then, **Mother** is implicitly Σ -definable under $\mathcal{O}$ and $C = \top$, for $\Sigma = \{\text{hasChild}, \text{Woman}\}$. Indeed, in any model $\mathcal{I}$ of $\mathcal{O}$, any element that satisfies **Woman** and has an **hasChild**-successor has to satisfy **Mother**, and other elements can not satisfy **Mother**. This is equivalent with saying that $\text{Woman} \sqcap \exists \text{hasChild}.\top$ is an explicit $\mathcal{ALC}(\Sigma)$ -definition of **Mother** under $\mathcal{O}$. $\square$

It should be clear that, if a concept is explicitly $\mathcal{L}(\Sigma)$ -definable under $\mathcal{O}$ and C , then it is implicitly Σ -definable under $\mathcal{O}$ and C , for any language $\mathcal{L}$. A logic enjoys the projective Beth definability property if the converse implication holds as well.

► **Definition 28.** *A DL $\mathcal{L}$ enjoys the projective Beth definability property (PBDP) if for any $\mathcal{L}$ ontology $\mathcal{O}$, $\mathcal{L}$ concepts C and C_0 , and signature $\Sigma \subseteq \text{sig}(C, \mathcal{O})$ the following holds: if C_0 is implicitly Σ -definable under $\mathcal{O}$ and C , then C_0 is explicitly $\mathcal{L}(\Sigma)$ -definable under $\mathcal{O}$ and C .*

Explicit definability is, similarly to interpolant existence, characterized via joint consistency.

28 Interpolation in Knowledge Representation

► **Lemma 29.** *Let $\mathcal{L}$ be any DL. Let C_0, C be $\mathcal{L}$ concepts, $\mathcal{O}$ an $\mathcal{L}$ ontology, and Σ be a signature. Then the following conditions are equivalent:*

1. *there is no explicit $\mathcal{L}(\Sigma)$ -definition for C_0 under $\mathcal{O}$ and C ;*
2. *$C \sqcap C_0$ and $C \sqcap \neg C_0$ are jointly $\sim_{\mathcal{L}, \Sigma}$ -consistent under $\mathcal{O}$.*

Based on Lemma 29, one can prove the equivalence of PBDP and the CIP.

► **Lemma 30.** *For every extension $\mathcal{L}$ of $\mathcal{ALC}$ with any of $\mathcal{S}, \mathcal{I}, \mathcal{F}, \mathcal{O}, \mathcal{H}$, $\mathcal{L}$ enjoys the CIP iff $\mathcal{L}$ enjoys the PBDP. Moreover, there are polynomial time reductions between computing interpolants and computing explicit definitions.*

Proof. The proof of “ $\Rightarrow$ ” is by a standard argument [37]: Assume that an $\mathcal{L}$ concept C_0 is implicitly Σ -definable under an $\mathcal{L}$ ontology $\mathcal{O}$ and $\mathcal{L}$ concept C , for some signature Σ . Then (14) holds. Take an $\mathcal{L}(\Sigma)$ -interpolant I for $C \sqcap C_0$ and $C_\Sigma \rightarrow C_{0\Sigma}$ under $\mathcal{O} \cup \mathcal{O}_\Sigma$. Then I is an explicit $\mathcal{L}(\Sigma)$ -definition of C_0 under $\mathcal{O}$ and C .

For “ $\Leftarrow$ ”, suppose $\mathcal{O}_1 \cup \mathcal{O}_2 \models C \sqsubseteq D$, for $\mathcal{L}$ ontologies $\mathcal{O}_1, \mathcal{O}_2$ and $\mathcal{L}$ concepts C, D , and let $\Sigma = \text{sig}(\mathcal{O}_1, C_1) \cap \text{sig}(\mathcal{O}_2, C_2)$. Based on Lemmas 19 and 29, one can show that there is a Craig interpolant for C and D under $\mathcal{O}_1 \cup \mathcal{O}_2$ iff there is an explicit $\mathcal{L}(\Sigma)$ -definition of D under $D \rightarrow C$ and $\mathcal{O}_1 \cup \mathcal{O}_2$. Finally observe that D is implicitly Σ -definable under $\mathcal{O}_1 \cup \mathcal{O}_2$ and $D \rightarrow C$: The right-hand side of the concept inclusion in (14) is $(D_\Sigma \rightarrow C_\Sigma) \rightarrow D_\Sigma$, a tautology. ◀

Lemma 30 is remarkable since for many logics the CIP is strictly stronger than the PBDP, see Chapter 3 and for example [98]. On the one hand, the proof of “ $\Rightarrow$ ” of Lemma 30 is rather robust and also applies to weaker logics, such as $\mathcal{EL}$, and weaker versions of explicit/implicit definability, such as the one with $C = \top$. On the other hand, the proof of “ $\Leftarrow$ ” relies on both the presence of the context concept C and on the expressive power of $\mathcal{ALC}$. In fact, we conjecture that the “ $\Leftarrow$ ”-direction does not hold for the weaker definition with $C = \top$.

Taking into account our knowledge about the CIP from Theorems 18 and 22, we obtain:

► **Corollary 31.** *$\mathcal{ALC}$ and its extensions by $\mathcal{S}, \mathcal{I}, \mathcal{F}$ enjoy the PBDP, but $\mathcal{ALCO}$ and $\mathcal{ALCH}$ and their extensions by $\mathcal{S}, \mathcal{I}, \mathcal{F}$ do not.*

We conclude the section with a brief discussion of two weaker forms of Beth definability that have also been considered in the DL literature.

- In the *non-projective Beth definability property (BDP)*, we are only interested in defining concept names $C_0 = A$ in terms of all other symbols in the signature, that is, $\Sigma = \text{sig}(\mathcal{O}) \setminus \{A\}$. Clearly, the PBDP implies the BDP, but the other direction is not true in general. For example, $\mathcal{ALCH}$ enjoys the BDP, but it lacks the PBDP. $\mathcal{ALCO}$ does not enjoy already BDP. We refer the reader to [8] for a broader discussion.

- In the *concept-based Beth definability property (CBDP)* both implicit and explicit definability are defined in terms of signatures which always contain all relevant role names, that is, only concept names may be restricted. In the same way, one can define a corresponding *concept Craig interpolation property (CCIP)*, and it has been shown recently using a sequent calculus that the highly expressive DL $\mathcal{RIQ}$ enjoys both CBDP and CCIP [97].

4.3 Applications

We discuss several applications of interpolants and explicit definitions, starting with a recently discovered relation of Craig interpolants to *description logic concept learning* in the presence of nominals. Informally, concept learning is the task of inducing a concept description from sets of positive and negative data examples, which has received a lot of interest in recent years, see e.g. [83, 66, 36]. The predominant application scenario is reverse-engineering of concepts to support ontology engineers in writing complex concepts. To make the connection to interpolation precise, we introduce some notation.

In our context, an *example* is a pair $(\mathcal{D}, a_0)$, where $\mathcal{D}$ is a database, that is, a finite set of facts of the form $A(a)$ and $r(a, b)$ with A a concept name, r a role name, and a, b individuals, and a_0 is an individual from database $\mathcal{D}$. A *labeled data set* is a pair (P, N) of sets of positive and negative examples. Let $\mathcal{L}$ be some DL, $\mathcal{O}$ be an $\mathcal{L}$ ontology, Σ a signature, and (P, N) a labeled dataset. An $\mathcal{L}(\Sigma)$ -separator for $\mathcal{O}, P, N$ is an $\mathcal{L}(\Sigma)$ concept C such that

- $\mathcal{O} \cup \mathcal{D} \models C(a)$ for all $(\mathcal{D}, a) \in P$ and
- $\mathcal{O} \cup \mathcal{D} \models \neg C(a)$ for all $(\mathcal{D}, a) \in N$.

Intuitively, a separator is a potentially complex concept that distinguishes between the positive and negative examples in the sense of consistent learning. We refer to the induced problem of deciding the existence of an $\mathcal{L}(\Sigma)$ -separator for given $\mathcal{O}, \Sigma, P, N$ with *$\mathcal{L}$ -separator existence*.

It has been recently observed that there is a strong connection between interpolants and separators in many DLs extending $\mathcal{ALCO}$. More precisely, for such DLs $\mathcal{L}$ there are polynomial time reductions between $\mathcal{L}$ -interpolant existence and $\mathcal{L}$ -separator existence. Moreover, the reductions guarantee that interpolants can be used as separators and vice versa [8, Theorem 4.3]. The presence of nominals is crucial in these reductions as they are needed to encode the database. Theorem 24 implies that separator existence in $\mathcal{ALCO}$ and its extensions is a hard yet decidable problem.

A special case of separator existence is the existence of *referring expressions*. Referring expressions originate in linguistics to refer to a single object in the domain of discourse [24], but have recently been introduced in data management and KR [7, 80, 19, 9]. Existence of a referring expression for object a can be cast as a separator existence problem, in which the task is to find a separator between a and all other individuals in the domain of discourse. Indeed, in many cases, the concrete individual, say the internal id $p12345$, could be less meaningful to the user than a concise description in terms of a given signature, e.g., *head of finance*. Referring expressions are also a special case of explicit definitions.

30 Interpolation in Knowledge Representation

Consider next the process of *ontology construction*. It offers essentially two ways to extend a present ontology by a new concept name, say A [118]. First, in the *explicit* manner, one adds a concept definition $A \equiv C$ for some concept description C over the signature of the present ontology. Second, in the *implicit* manner, one can add (several) concept inclusions with the property that in every model of the ontology, the interpretation of A is uniquely determined by the interpretation of the other symbols—in other words: A is implicitly defined. Of course, the explicit way is more transparent and thus to be preferred. However, if the ontology language of choice enjoys the PBDP, there is actually an explicit definition for A , also in the second case. The notion of PBDP has been studied in this sense under the disguise of *definitorial completeness* in [12, 118].

Finally, we discuss ontology-mediated query answering (OMQ). Recall that in OMQ we access a knowledge base consisting of ontology $\mathcal{O}$ and database $\mathcal{D}$ using a query $\varphi(\vec{x})$ with the goal to determine the *certain answers*, that is, all tuples $\vec{a}$ such that $\mathcal{O} \cup \mathcal{D} \models \varphi(\vec{a})$. Depending on the ontology language, this is a potentially difficult entailment problem and the question is whether one can *rewrite* φ and $\mathcal{O}$ into $\varphi_{\mathcal{O}}$ such that $\mathcal{O} \cup \mathcal{D} \models \varphi(\vec{a})$ iff $\mathcal{D} \models \varphi_{\mathcal{O}}(\vec{a})$, for all $\vec{a}$. Note that in this case $\varphi_{\mathcal{O}}$ can be answered efficiently by a standard database system. The PBDP and the connection to interpolation from Lemma 30 have been used several times to study the existence and computation of such rewritings [116, 34, 35, 121]. Chapter 14 discusses further applications of the PBDP in the context of database query answering.

4.4 Computing Interpolants

In the previous sections, we have motivated the need for computing interpolants and explicit definitions in various applications. Unfortunately, in spite of that, we are not aware of any DL reasoner that supports the actual computation of interpolants. Hence, this section is restricted to the known theoretical results. By Lemma 30, we can focus on computing interpolants. We concentrate on extensions of $\mathcal{ALC}$ that enjoy the CIP and refer the reader to [33] for $\mathcal{EL}$ and its relatives. In the following theorem from [119], we refer to a DAG-representation in which each subconcept is represented only once as *succinct representation* of concepts.

► **Theorem 32** (Theorems 3.10, 3.26 in [119]). *Let $\mathcal{L}$ be $\mathcal{ALC}$ or any extension with $\mathcal{S}, \mathcal{I}, \mathcal{F}$. Then, one can compute, given $\mathcal{L}$ ontologies $\mathcal{O}_1, \mathcal{O}_2$ and $\mathcal{L}$ concepts C_1, C_2 with $\mathcal{O}_1 \cup \mathcal{O}_2 \models C_1 \sqsubseteq C_2$, in double exponential time a Craig interpolant of C_1 and C_2 under $\mathcal{O}_1 \cup \mathcal{O}_2$. If concepts are represented succinctly, then the interpolant can be computed in exponential time.*

In the same paper, it is shown that the statement in Theorem 32 is essentially optimal. We give the lower bound in terms of explicit definitions; it transfers to Craig interpolants via (the proof of) Lemma 30. It is also not difficult to see that it remains valid when we consider extensions of $\mathcal{ALC}$ with $\mathcal{S}, \mathcal{I}, \mathcal{F}$.

► **Theorem 33** (Theorem 4.11 in [119]). *Let $\Sigma = \{r, s\}$ consist of two role names. For every $n \geq 0$, there are an $\mathcal{ALC}$ concept C_n and an $\mathcal{ALC}$ ontology $\mathcal{O}_n$ of size polynomial in n such*

that $\Sigma \subseteq \text{sig}(\mathcal{O}_n, C_n)$ and C_n is implicitly Σ -definable under $\mathcal{O}_n$, but the smallest explicit $\mathcal{ALC}(\Sigma)$ -definition of C_n under $\mathcal{O}_n$ is double exponentially long in n .

Proof sketch. The proof is via a well-known *path-set construction* [91]. The idea is that C_n together with $\mathcal{O}_n$ enforces a full binary r/s -tree of depth 2^n . One then shows that any $\mathcal{ALC}(\Sigma)$ concept D that explicitly defines such a tree is double exponentially long in n , since each path in the tree has to be reflected in a (different) path in the syntax tree of D . ◀

The original proof of Theorem 32 is by an extension of the tableau-based proof of the CIP for the respective DLs in [119]. Instead of giving it here, we provide two different perspectives on the computation problem, focusing on $\mathcal{ALC}$ for the sake of simplicity. The first one is a reduction of computing interpolants under ontologies to the case without ontologies. The second one is a refined analysis of the mosaic elimination algorithm that decides joint $\sim_{\mathcal{ALC}, \Sigma}$ -consistency given in Section 4.1. The benefit of the former is that this enables us to use all methods for computing interpolants in modal logic presented in Chapter 4. The benefit of the latter is that it provides the same upper bounds as claimed in Theorem 32.

Perspective 1: The Reduction

The main idea is to *materialize* the ontology to sufficient depth. To formalize this, let $\mathcal{ALC}_n(\Sigma)$ denote the set of all $\mathcal{ALC}(\Sigma)$ concepts of role depth at most n , and define for each $\mathcal{ALC}$ ontology $\mathcal{O}$ using role names Σ_R and each $n \geq 0$, a concept $\Phi_{\mathcal{O}}^n \in \mathcal{ALC}_n(\Sigma)$ by taking

$$\Phi_{\mathcal{O}}^n = \bigcap_{m \leq n} \bigcap_{r_1, \dots, r_m \in \Sigma_R} \forall r_1 \dots \forall r_m. \bigcap_{C \sqsubseteq D \in \mathcal{O}} (\neg C \sqcup D).$$

Intuitively, the concept $\Phi_{\mathcal{O}}^n$ expresses that the concept inclusions in $\mathcal{O}$ are satisfied along any Σ_R -path of length at most n . The following lemma, whose proof is similar to the proof of Lemma 9 in [118], provides the promised reduction.

► **Lemma 34.** *Let $\mathcal{O}$ be an $\mathcal{ALC}$ ontology, C_1, C_2 be $\mathcal{ALC}$ concepts, and $N > 2^{||\mathcal{O}|| + ||C_1|| + ||C_2||}$. Then, for all signatures Σ , all $n \geq 0$, and all $I \in \mathcal{ALC}_n(\Sigma)$ the following are equivalent:*

1. *I is an $\mathcal{ALC}(\Sigma)$ -interpolant of C_1 and C_2 under $\mathcal{O}$;*
2. *I is an $\mathcal{ALC}(\Sigma)$ -interpolant of $\Phi_{\mathcal{O}}^{N+n} \sqcap C_1$ and $\Phi_{\mathcal{O}}^{N+n} \rightarrow C_2$.*

Thus, if we were interested in (Craig) interpolants of a certain role depth n , we could use Lemma 34 to compute them using methods for modal logic from Chapter 4. Since Theorem 32 entails an exponential upper bound on the role depth of interpolants (an interpolant that is of exponential size in succinct representation has at most exponential role depth), this is also a complete approach for computing Craig interpolants. We refrain from analyzing the size of interpolants constructed via this method.

32 Interpolation in Knowledge Representation

Perspective 2: Construction from Mosaic Elimination

We conduct a refined analysis of the mosaic elimination algorithm that decides joint consistency in $\mathcal{ALC}$ in Section 4.1. Let $\mathcal{O}, C_1, C_2, \Sigma$ be an input to mosaic elimination. Recall that the elimination algorithm computes a sequence $\mathcal{M}_0, \mathcal{M}_1, \dots$ of sets of mosaics, where $\mathcal{M}_0$ is the set of all mosaics, and $\mathcal{M}_{i+1}$ is obtained from $\mathcal{M}_i$ by eliminating bad mosaics. We say that a mosaic (t_1, t_2) is *eliminated in round ℓ* if $(t_1, t_2) \in \mathcal{M}_\ell$ but $(t_1, t_2) \notin \mathcal{M}_{\ell+1}$. In a slight abuse of notation, we treat a type t as the concept $\bigcap_{D \in t} D$.

► **Lemma 35.** *If a mosaic (t_1, t_2) is eliminated in round ℓ , then there is an $\mathcal{ALC}_\ell(\Sigma)$ concept I such that $\mathcal{O} \models t_1 \sqsubseteq I$ and $\mathcal{O} \models t_2 \sqsubseteq \neg I$.*

Proof. The proof is by induction on ℓ . For the base case, let $\ell = 0$. If (t_1, t_2) is eliminated in round 0, then this is due to the violation of atomic consistency. Then we can choose $I = A$ or $I = \neg A$ depending on whether the problematic concept name A is contained in t_1 or in t_2 .

For the inductive case, let $\ell > 0$. Then (t_1, t_2) has been eliminated due to violation of existential saturation, that is, there is $r \in \Sigma$, $i \in \{1, 2\}$, and $\exists r.C \in t_i$ such that each $(t'_1, t'_2) \in \mathcal{M}_i$ with $C \in t_i$ satisfies $(t_1, t_2) \not\prec_r (t'_1, t'_2)$. Assume first that $i = 1$, and let T_1, T_2 be the following sets of types:

$$T_1 = \{t \mid C \in t \text{ and } t_1 \rightsquigarrow_r t\} \quad \text{and} \quad T_2 = \{t \mid t_2 \rightsquigarrow_r t\}.$$

By induction hypothesis, for each $(t'_1, t'_2) \in T_1 \times T_2$ there is an $\mathcal{ALC}_{\ell-1}(\Sigma)$ concept $I_{t'_1, t'_2}$ such that $\mathcal{O} \models t'_1 \sqsubseteq I_{t'_1, t'_2}$ and $\mathcal{O} \models t'_2 \sqsubseteq \neg I_{t'_1, t'_2}$. We choose

$$I = \exists r. \bigsqcup_{t'_1 \in T_1} \prod_{t'_2 \in T_2} I_{t'_1, t'_2}.$$

Clearly, $I \in \mathcal{ALC}_\ell(\Sigma)$. We verify $\mathcal{O} \models t_1 \sqsubseteq I$ and $\mathcal{O} \models t_2 \sqsubseteq \neg I$.

- For the former, let $\mathcal{I}$ be a model of $\mathcal{O}$ and $d \in t_1^\mathcal{I}$. Since $\exists r.C \in t_1$, there is some $e \in C^\mathcal{I}$ with $(d, e) \in r^\mathcal{I}$. Let $t'_1 = \text{tp}_\mathcal{I}(e)$ and note that $t'_1 \in T_1$. By what was said above, we have $\mathcal{O} \models t'_1 \sqsubseteq I_{t'_1, t'_2}$ for all $t'_2 \in T_2$. Thus, $d \in (\exists r. I_{t'_1, t'_2})^\mathcal{I}$ for all $t'_2 \in T_2$, and hence $d \in I^\mathcal{I}$.
- For the latter, let $\mathcal{I}$ be a model of $\mathcal{O}$ and $d \in t_2^\mathcal{I}$. We have to show that $d \notin I^\mathcal{I}$, that is, $d \in (\forall r. \prod_{t'_1 \in T_1} \bigsqcup_{t'_2 \in T_2} \neg I_{t'_1, t'_2})^\mathcal{I}$. Suppose e is an r -successor of d and let $t'_2 = \text{tp}_\mathcal{I}(e)$. Note that $t'_2 \in T_2$. By what was said above, we have $e \in (\neg I_{t'_1, t'_2})^\mathcal{I}$ for all $t'_1 \in T_1$. Thus $d \notin I^\mathcal{I}$.

The case $i = 2$ is dual; the interpolant has a leading universal quantifier then. ◀

Using the same arguments, one can finally show the following:

► **Lemma 36.** *For $i = 1, 2$, let T_i be the set of all types that contain C_i . If the result $\mathcal{M}^*$ of the mosaic elimination does not contain any mosaic from $T_1 \times T_2$, then*

$$I = \bigsqcup_{t_1 \in T_1} \prod_{t_2 \in T_2} I_{t_1, t_2}$$

is an $\mathcal{ALC}_N(\Sigma)$ -interpolant of C_1 and $\neg C_2$ under $\mathcal{O}$, for $N \leq 2^{||\mathcal{O}||+||C_1||+||C_2||}$.

A straightforward analysis shows that, moreover, the size of interpolants I constructed in this way is at most double exponential in the size of the input $\mathcal{O}, C_1, C_2, \Sigma$, and at most exponential if we allow for succinct representation. Thus, this approach achieves the upper bounds claimed in Theorem 32.

We conjecture that it is rather straightforward to extend this way of constructing interpolants to extensions of $\mathcal{ALC}$ enjoying the CIP. It appears to be much more challenging for the extension with $\mathcal{H}$ and/or $\mathcal{O}$, which lead to the loss of the CIP. Some progress has recently been made for the extension with $\mathcal{H}$ in [63], which also reports that an earlier algorithm for computing interpolants under $\mathcal{ALCH}$ ontologies from [8] is not correct (and not easy to fix).

5 Interpolation in Logic Programming

In this section, we briefly discuss interpolation in the context of Logic Programming (LP) [13], where applications of interpolation align with the ones discussed in Section 1. Here, we focus on logic programs under the stable model semantics [45], as this arguably is the most widely used semantics in LP. In particular, it is the semantics employed in Answer Set Programming (ASP) [43, 86], a form of declarative programming aimed at solving large combinatorial (commonly NP-hard) search problems. In addition, a large part of the research in LP on interpolation and, in particular, on the problem of forgetting focusses on ASP, motivated by its applications.⁵

5.1 Logic Programs and Answer Sets

We start by recalling necessary notions and notation for logic programs [43, 86].

We consider first-order signatures Σ consisting of mutually disjoint and countably infinite sets of constants, functions, predicates, and variables. This allows us to define *terms* as usual as constants, variables, and complex terms $f(t_1, \dots, t_n)$, where f is a function and $t_1, \dots, t_n$ are terms, as well as *atoms* $p(t_1, \dots, t_n)$ where p is a predicate and $t_1, \dots, t_n$ are terms. We denote the set of all atoms over Σ with $A(\Sigma)$.

Given Σ , a (*logic*) *program* P is a finite set of *rules* r of the form⁶

$$a_1 \vee \dots \vee a_k \leftarrow b_1, \dots, b_l, \text{not } c_1, \dots, \text{not } c_m, \text{not not } d_1, \dots, \text{not not } d_n \quad (15)$$

where $\{a_1, \dots, a_k, b_1, \dots, b_l, c_1, \dots, c_m, d_1, \dots, d_n\} \subseteq A(\Sigma)$. We also represent such rules with

$$H \leftarrow B^+, \text{not } B^-, \text{not not } B^{--}, \quad (16)$$

⁵ Stable models and answer sets are commonly used synonymously. Though the former is more associated with semantics and the latter with problem solving, the latter has gained wider adoption [85, 22].

⁶ We admit double negation in rules which is historically less common, but needed in our context.

34 Interpolation in Knowledge Representation

where $H = \{a_1, \dots, a_k\}$, $B^+ = \{b_1, \dots, b_l\}$, $B^- = \{c_1, \dots, c_m\}$, and $B^{--} = \{d_1, \dots, d_n\}$. We distinguish the *head* of r , $\text{head}(r) = H$, and its *body*, $\text{body}(r) = B^+ \cup \text{not } B^- \cup \text{not not } B^{--}$, where $\text{not } B^-$ and $\text{not not } B^{--}$ represent $\{\text{not } p \mid p \in B^-\}$ and $\{\text{not not } p \mid p \in B^{--}\}$, respectively. We refer to the set of all (logic) programs as the *class of (logic) programs*. Different more specific kinds of rules exist: if $n = 0$, then r is *disjunctive*; if also $k \leq 1$, then r is *normal*; if in addition $m = 0$, then r is *Horn*, if, moreover, $l = 0$, then r is a *fact*. The class of *disjunctive programs* is defined as a set of finite sets of disjunctive rules, and other classes can be defined accordingly.

For the semantics, we focus on programs in which variables have been instantiated. Given a program P , this is captured by a ground program, denoted $\text{ground}(P)$, which is obtained from P by replacing the variables in P with ground terms over Σ in all possible ways. We also call variable-free terms, atoms, rules, and programs *ground*. Accordingly, we consider the set of all ground atoms over Σ with $A_{gr}(\Sigma)$. Note that ground programs correspond to propositional programs, where the signature is restricted to a countably infinite set of propositional variables, which can be viewed as atoms built over predicates with arity 0, i.e., no terms, in which case Σ would simplify to such a set of nullary predicates.

We define answer sets [46], building on HT-models from the logic of here-and-there [58], the monotonic logic underlying answer sets [107]. First, we recall the *reduct* P^I of a ground program P with respect to a set I of ground atoms, adapted here to extended rules:

$$P^I = \{H \leftarrow B^+ \mid r \text{ of the form (16) in } P \text{ such that } B^- \cap I = \emptyset \text{ and } B^{--} \subseteq I\}.$$

The idea is, assuming the atoms in I as true, to transform (ground) P into a corresponding program P^I that does not contain negation. Then, HT-models are defined as a pair of sets of (ground) atoms, making use of a standard entailment relation $\models$ for programs, where (ground) programs are viewed as a conjunction of implications corresponding to the rules of the form (15) with $\neg$, $\wedge$, and $\vee$ denoting classical negation, conjunction, and disjunction, respectively. Intuitively, in an HT-interpretation $\langle X, Y \rangle$, Y is used to assess satisfaction of P as such, and X is employed to verify satisfaction of the reduct P^Y . An answer set is then a minimal model that satisfies the reduct with respect to itself.

Formally, let P be a ground program. An *HT-interpretation* is a pair $\langle X, Y \rangle$ such that $X \subseteq Y \subseteq A_{gr}(\Sigma)$, and $\langle X, Y \rangle$ is an *HT-model* of P if $Y \models P$ and $X \models P^Y$. The set of *all HT-models* of P is written $\mathcal{HT}(P)$, and P is *satisfiable* if $\mathcal{HT}(P) \neq \emptyset$. A set of (ground) atoms Y is an *answer set* of P if $\langle Y, Y \rangle \in \mathcal{HT}(P)$, and there is no $X \subsetneq Y$ such that $\langle X, Y \rangle \in \mathcal{HT}(P)$. We denote by $\mathcal{AS}(P)$ the set of all answer sets of P , and call P *coherent* if $\mathcal{AS}(P) \neq \emptyset$.

We use two entailment relations between logic programs P_1, P_2 for the two introduced model notions, allowing to specify that P_1 entails P_2 : namely, $\models_{\text{HT}}$ represents entailment for HT logic, and $\vdash_c$ represents cautious entailment for answer sets. Formally, given programs P_1 and P_2 , $P_1 \models_{\text{HT}} P_2$ holds if $\mathcal{HT}(P_1) \subseteq \mathcal{HT}(P_2)$, and $P_1 \vdash_c P_2$ holds if $\mathcal{AS}(P_1) \subseteq \mathcal{AS}(P_2)$. Note that this also allows us to express that program P_1 entails a fact, a set of facts, or a disjunction, etc, as P_2 allows to represent these.

► **Example 37.** Consider for simplicity the following (propositional) program P :

$$a \leftarrow \text{not } b \qquad b \leftarrow \text{not } c \qquad e \leftarrow d \qquad d \leftarrow a$$

Then $\langle b, bde \rangle$ is an HT-model of P because $\{b, d, e\} \models P$ and $\{b\} \models P^{\{b, d, e\}}$ where $P^{\{b, d, e\}} = \{b \leftarrow; e \leftarrow d; d \leftarrow a\}$. Hence, $\{b, d, e\}$ is not an answer set of P as it violates the minimality condition. Moreover, $\{b, d\}$ is not an answer set of P because $\{b, d\} \not\models P$. In fact, we can verify that $\{b\}$ is its only answer set. Hence, $b \leftarrow$ is entailed by P with respect to $\sim_c$. $\square$

5.2 Craig Interpolants

In nonmonotonic formalisms, including Logic Programming, previously drawn conclusions may become invalid in the presence of new information. For example, if we were to add a rule $c \leftarrow$ to the program in Example 37, then $\{b\}$ would no longer be an answer set. Instead, the unique answer set would be $\{a, d, e\}$. This raises problems for (Craig) interpolation for logic programs, as simply replacing the entailment relation $\models$ in Definition 1 with a nonmonotonic entailment relation $\sim$, which is not transitive in general, may not work.

Following ideas in the literature [37], Gabbay et al. [38] proposed to adjust the central condition of Craig interpolants to logic programs using two entailment relations. We recall this generalized definition of interpolants using two abstract entailment relations.

► **Definition 38.** Let $\vdash_1$ and $\vdash_2$ be entailment relations and ϕ, ψ programs such that $\phi \sim \psi$. A $(\vdash_1, \vdash_2)$ interpolant for (ϕ, ψ) is a program χ such that $\phi \vdash_1 \chi$ and $\chi \vdash_2 \psi$, and χ uses only non-logical symbols occurring in both ϕ and ψ .

This is motivated by the aim to combine a nonmonotonic entailment relation $\sim$ with its deductive base, i.e., a logic L with monotonic entailment relation $\vdash_L$ such that (i) $\vdash_L \subseteq \sim$; (ii) for programs ϕ_1, ϕ_2 , if $\phi_1 \equiv_L \phi_2$, then $\phi_1 \approx \phi_2$ (i.e., ϕ_1 and ϕ_2 have the same nonmonotonic entailments); and (iii) if $\phi \sim \chi$ and $\chi \vdash_L \psi$, then $\phi \sim \psi$.

Two kinds of Craig interpolants are defined. The stronger one $(\sim, \vdash_L)$ builds on this notion of deductive base, which guarantees among other things that the relation $(\sim, \vdash_L)$ is transitive in the sense of (iii). The weaker one, $(\sim, \sim)$, is implied by the former, but does not guarantee transitivity per se.

Also, two variants of the entailment relation $\sim_c$ are considered, one which is aligned with closed world assumption (CWA), i.e., atoms that cannot be proven true are considered false, and one which is aligned with open world assumption (OWA), where no such conclusions on false atoms can be drawn.

In the latter case, it is possible to show for propositional programs that $(\sim_c, \models_{\text{HT}})$ interpolants exist by constructing a representation of the minimal models of ϕ , taking advantage of the fact that HT logic serves as deductive base for answer set semantics and that Craig interpolants as per Definition 1 are guaranteed to exist for HT logic [37].

However, ASP employs CWA, and in this case, in general, only $(\sim_c, \sim_c)$ interpolants do exist. Here, the corresponding model representation can be obtained because, in addition to

36 Interpolation in Knowledge Representation

the previously mentioned points, under CWA, the non-logical symbols occurring in ψ , but not in ϕ , can be considered false. This also ensures, among others, transitivity of $(\vdash_c, \vdash_c)$ [38].

► **Theorem 39.** *For coherent propositional programs, $(\vdash_c, \vdash_c)$ interpolants are guaranteed to exist.*

The stronger notion of $(\vdash_c, \models_{HT})$ interpolants only holds for coherent propositional programs under CWA when the non-logical symbols of ψ are a subset of those of ϕ [38].

However, in the first-order case, such interpolants may not exist in general, as the formula representing the answer sets may not be first-order definable. Yet, first-order definability can be ensured for programs where function symbols are not allowed and where variables are safe. Here, *safe* refers to all variables in a rule having to appear in atoms in B^+ in (16), which ensures that any grounding is finite as well.⁷ This restriction is not problematic in practice in the context of ASP with variables, as programs are required to be safe anyway for the grounding, employed in state-of-the-art ASP solvers, such as clingo [44] or DLV2 [4], where the usage of function symbols is also severely restricted to avoid grounding problems.

► **Theorem 40.** *For coherent, function-free, and safe first-order programs, $(\vdash_c, \vdash_c)$ interpolants are guaranteed to exist.*

5.3 Uniform Interpolation and Forgetting

As mentioned in Section 1, uniform interpolation and forgetting are strongly connected in their aim to reduce the language while preserving information over the remaining language. However, in the context of ASP, mainly forgetting has drawn considerable attention, varying from early purely syntactic approaches to a number of approaches using different means to capture the idea of preserving information over the remaining language [51]. In the following, we briefly discuss the main results and their implications on uniform interpolants.

In accordance with the literature, and to ease presentation, we focus on the propositional case. Hence, the signature Σ simplifies to propositional variables, i.e., only predicates of arity 0. We start by refining the definition of uniform interpolants in this context, as before admitting a generic entailment relation $\vdash$.

► **Definition 41.** *Let $V \subseteq \Sigma$ be a set of atoms, P a propositional program, and $\vdash$ an entailment relation. A uniform $(V, \vdash)$ -interpolant for P is a program P_V such that*

- $P \vdash P_V$;
- P_V uses only atoms from V ; and
- for any program P' using only atoms from V , if $P \vdash P'$, then $P_V \vdash P'$.

⁷ In fact, a slightly more general notion of safe is used to capture a wider range of ASP constructs [38].

Note that for arbitrary nonmonotonic entailment relations, the first condition may in general not hold. But, building on results from forgetting, such interpolants can indeed be found.

For forgetting, the focus is on operators that return a program over the remaining language. Here, unlike uniform interpolation, the atoms to be forgotten are explicitly mentioned.

► **Definition 42.** *Given a class of logic programs $\mathcal{C}$ over Σ , a forgetting operator (over $\mathcal{C}$) is defined as a function $f : \mathcal{C} \times 2^\Sigma \rightarrow \mathcal{C}$ where, for each $P \in \mathcal{C}$ and $V \subseteq \Sigma$, $f(P, V)$, the result of forgetting about V from P , is a program over $\Sigma(P) \setminus V$.*

In the literature (see, e.g., [51]), operators then aim to preserve the semantic relations between atoms in $\Sigma(P) \setminus V$ from P , such as the dependencies discussed in the following example.

► **Example 43.** Consider again program P from Example 37. If we want to forget about d from P , then the first two rules should remain unchanged, while the latter two should not occur in the result. At the same time, implicit relations, such as e depending on a via d , should be preserved. Hence, we would expect the result $\{a \leftarrow \text{not } b; b \leftarrow \text{not } c; e \leftarrow a\}$. Alternatively, consider forgetting about b from P . Now, the final two rules remain, whereas the first two would be replaced by one linking a and c , i.e., $\{a \leftarrow \text{not not } c; e \leftarrow d; d \leftarrow a\}$. ◻

Note that, in the second part, we cannot simply use the rule $a \leftarrow c$ instead, absorbing the double negation, as this is not equivalent in general (in terms of HT-models). Thus, results of forgetting may be required to be in a class more general than the one of the given program.

Example 43 illustrates the main idea of such semantic relations to be preserved, but leaves a formal specification open. Such characterization is usually semantic, i.e., a characterization of the desired models, which does not fix a specific program, and hence cannot align with a single function. This is captured by classes of forgetting operators that conjoin several operators based on a common characterization. Here, since we are interested in the existence of interpolants, we focus on individual operators instead (simplifying some technical material).

A number of different operators and classes of these have been defined in the literature, as well as certain properties with the aim of aiding the semantic characterization. We recall relevant properties in the context of interpolation and refer for a full account to [51].

We start with a property (for forgetting operators f) comparing the answer sets of a program and its result of forgetting. For that, given a set of atoms V , the V -exclusion of a set of answer sets $\mathcal{M}$, written $\mathcal{M}_{\parallel V}$, is defined as $\{X \setminus V \mid X \in \mathcal{M}\}$.

(CP) f satisfies *Consequence Persistence* if, for each program P and $V \subseteq \Sigma$, we have $\mathcal{AS}(f(P, V)) = \mathcal{AS}(P)_{\parallel V}$.

Essentially, (CP) requires that answer sets of the forgetting result be answer sets of the original program projected to the remaining language and vice-versa [124]. There are operators that satisfy (CP), e.g., those in class F_{SM} [124], whose forgetting results are characterized as the subset of HT-models (projected to the remaining language) such that the condition for (CP) holds. We can show that, if f satisfies (CP), then its forgetting result $f(P, V)$ provides a uniform $(V, \vdash_c)$ -interpolant.

38 Interpolation in Knowledge Representation

► **Theorem 44.** *For propositional programs P and $V \subseteq \Sigma$, uniform $(V, \sim_c)$ -interpolants are guaranteed to exist.*

The intuition here is that, since **(CP)** holds, we have $P \sim_c f(P, V)$ and, likewise, the third condition of Def. 41 holds. Note that, for the two programs in Example 43, both would constitute uniform $(V, \sim_c)$ -interpolants with $V = \{a, b, c, e\}$ in the first case and $V = \{a, c, d, e\}$ in the second.

Other properties rather focus on HT-models and entailments based on this notion [125].

(W) f satisfies *Weakening* if, for each program P and $V \subseteq \Sigma$, we have $P \models_{\text{HT}} f(P, V)$.

(PP) f satisfies *Positive Persistence* if, for each program P and $V \subseteq \Sigma$: if $P \models_{\text{HT}} P'$, with $\Sigma(P') \subseteq \Sigma \setminus V$, then $f(P, V) \models_{\text{HT}} P'$.

These properties are aligned with Definition 41, and based on them and classes that satisfy these, e.g., F_{HT} , we can also capture interpolants with respect to $\models_{\text{HT}}$ [126].

► **Theorem 45.** *For propositional programs P and $V \subseteq \Sigma$, uniform $(V, \models_{\text{HT}})$ -interpolants are guaranteed to exist.*

Here, again both programs in Example 43 are uniform $(V, \models_{\text{HT}})$ -interpolants. For a simple example where these two kinds of interpolants differ, consider $P = \{a \leftarrow \text{not } b; b \leftarrow \text{not } a\}$. Then, $\{a \leftarrow \text{not not } a\}$ is uniform $(a, \sim_c)$ -interpolant, but not a uniform $(a, \models_{\text{HT}})$ -interpolant. On the other hand $\{\}$ is a $(a, \models_{\text{HT}})$ -interpolant, but not a uniform $(a, \sim_c)$ -interpolant.

Determining whether a given program is a result of forgetting, and hence whether it is a uniform interpolant, is computationally expensive. It has been shown that, for F_{HT} and F_{SM} , this problem is Π_2^P -complete with respect to data complexity (where the size of the input is measured in the number of facts) [126, 124]. It has also been argued that, e.g., for F_{SM} the algorithm to compute a result may result in an exponential blow-up in the size of the overall program [124].

Stronger notions of preserving semantic relations have been considered in Forgetting [68].

(SP) f satisfies *Strong Persistence* if, for each program P , $V \subseteq \Sigma$, and program R with $\Sigma(R) \subseteq \Sigma \setminus V$, we have $\mathcal{AS}(f(P, V) \cup R) = \mathcal{AS}(P \cup R)_{\parallel V}$.

The idea is to preserve the answer sets no matter what other program is added to the given program and its result of forgetting. This turns out to be not possible in general [50], and concise conditions when such forgetting is possible and classes of operators approximating this have been investigated [52]. Complementarily, when the added programs R are restricted to sets of facts, forgetting is always possible with applications in modular ASP [49]. In both cases, it has been shown that determining whether a given program is a result of forgetting for these classes is more demanding, i.e., Π_3^P -complete (data complexity). This confirms that such notions of forgetting indeed go beyond uniform interpolation. Also, syntactic operators have been considered for these classes, but their construction may again, in the worst case,

result in an exponential blowup in the size of the program [16, 15, 48]. Corresponding algorithms of these, as well as those building on constructing a representative of the semantic characterization have been implemented in the Web tool ForgettingWeb [17].

One open problem for future research is how to tackle in general finding uniform interpolants/forgetting for first-order programs with variables in a wider sense (without requiring the grounding up-front). Possible avenues include extensions of the language of programs, in the line of the work by Aguado et al., which allows Strong Persistence to always hold [2]. Alternatively, connections to related notions on abstraction in ASP might be pursued, that rather aim at abstracting different elements of the language into a conjoined abstract one, either on the object level or in between different concepts [110, 111, 112, 113]. A further option arises from recent work where first-order Craig-interpolation is used to synthesize strongly equivalent programs given some background knowledge (in the form of a program) with an implementation using first-order reasoners [57].

6 Conclusion

We discussed interpolation in two relevant KR formalisms, DLs and logic programming. In the context of DLs, we focussed on uniform interpolation at the level of ontologies, and on Craig interpolation at the level of concepts. Uniform interpolation is strongly related to the topic of *forgetting*, which has been extensively studied in many settings. Interpolation has also been extensively studied for modal logics and classical logics, which are formalisms relevant for KR that are discussed in other chapters of this volume. Interestingly, there is comparably little research on interpolation in other formalisms relevant for KR.

With logic programs, we discussed one formalism with a nonmonotonic entailment relation. Nonmonotonic reasoning plays indeed a central role in many KR applications, and many more nonmonotonic KR formalisms have been proposed. Examples include *default logic* [18], *belief revision* [42], *argumentation frameworks* [14], as well as semantics based on *circumscription* [99] and on *repairs* [84]. Interpolation and the related notion of forgetting have been researched in some of these formalisms as well, but many problems remain open. The investigations in [39, Chapter 9], [114, Chapter 6] and [5] discuss interpolation in nonmonotonic logics on a more abstract level. Interpolation and Beth definability for default logics is discussed in [25], and forgetting for defeasible logic has been investigated in [6].

While there are implemented systems for computing uniform interpolants of DL ontologies, Craig interpolation and Beth definability have so far been investigated mostly on a theoretical level. Given the increased interest in explainability [101, 3] and supervised learning [106] for DLs in the recent years, which are relevant use cases of Craig interpolation, this situation might change in the future. Indeed, Craig interpolation at the level of ontologies, implemented via a reduction to uniform interpolation, is used in the ontology explanation tool EVEL to create so-called *elimination proofs* [3].

Acknowledgments

The authors would like to thank Michael Benedikt and Balder ten Cate for valuable comments on earlier versions of this chapter. M. Knorr acknowledges partial support by project BIO-REVISE (2023.13327.PEX) and by UID/04516/NOVA Laboratory for Computer Science and Informatics (NOVA LINCS) with the financial support of FCT/IP.

References

- 1 Wilhelm Ackermann. Zum Eliminationsproblem der mathematischen Logik. *Mathematische Annalen*, 111(1):61–63, 1935. doi:<https://doi.org/10.1007/BF01448035>.
- 2 Felicidad Aguado, Pedro Cabalar, Jorge Fandinno, David Pearce, Gilberto Pérez, and Concepción Vidal. Forgetting auxiliary atoms in forks. *Artif. Intell.*, 275:575–601, 2019. doi:[10.1016/J.ARTINT.2019.07.005](https://doi.org/10.1016/J.ARTINT.2019.07.005).
- 3 Christian Alrabbaa, Stefan Borgwardt, Tom Frieze, Anke Hirsch, Nina Knieriemen, Patrick Koopmann, Alisa Kovtunova, Antonio Krüger, Alexej Popovic, and Ida S. R. Siahaan. Explaining reasoning results for OWL ontologies with Eevee. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning, KR 2024*, 2024. doi:[10.24963/KR.2024/67](https://doi.org/10.24963/KR.2024/67).
- 4 Mario Alviano, Francesco Calimeri, Carmine Dodaro, Davide Fuscà, Nicola Leone, Simona Perri, Francesco Ricca, Pierfrancesco Veltri, and Jessica Zangari. The ASP system DLV2. In *Logic Programming and Nonmonotonic Reasoning - 14th International Conference, LPNMR 2017*, volume 10377 of *Lecture Notes in Computer Science*, pages 215–221. Springer, 2017. doi:[10.1007/978-3-319-61660-5_19](https://doi.org/10.1007/978-3-319-61660-5_19).
- 5 Eyal Amir. Interpolation theorems for nonmonotonic reasoning systems. In *Logics in Artificial Intelligence, European Conference, JELIA 2002*, volume 2424 of *Lecture Notes in Computer Science*, pages 233–244. Springer, 2002. doi:[10.1007/3-540-45757-7_20](https://doi.org/10.1007/3-540-45757-7_20).
- 6 Grigoris Antoniou, Thomas Eiter, and Kewen Wang. Forgetting for defeasible logic. In *Logic for Programming, Artificial Intelligence, and Reasoning - 18th International Conference, LPAR-18*, volume 7180 of *Lecture Notes in Computer Science*, pages 77–91. Springer, 2012. doi:[10.1007/978-3-642-28717-6_9](https://doi.org/10.1007/978-3-642-28717-6_9).
- 7 Carlos Areces, Alexander Koller, and Kristina Striegnitz. Referring expressions as formulas of description logic. In *Proceedings of the 5th International Natural Language Generation Conference, INLG 2008*. The Association for Computer Linguistics, 2008. URL: <https://aclanthology.org/W08-1107/>.
- 8 Alessandro Artale, Jean Christoph Jung, Andrea Mazzullo, Ana Ozaki, and Frank Wolter. Living without beth and craig: Definitions and interpolants in description and modal logics with nominals and role inclusions. *ACM Trans. Comput. Log.*, 24(4):34:1–34:51, 2023. doi:[10.1145/3597301](https://doi.org/10.1145/3597301).
- 9 Alessandro Artale, Andrea Mazzullo, Ana Ozaki, and Frank Wolter. On free description logics with definite descriptions. In *Proceedings of the 18th International Conference on Principles of Knowledge Representation and Reasoning, KR 2021*, pages 63–73, 2021. doi:[10.24963/kr.2021/7](https://doi.org/10.24963/kr.2021/7).

- 10 Franz Baader, Sebastian Brandt, and Carsten Lutz. Pushing the EL envelope. In *IJCAI-05, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence*, pages 364–369. Professional Book Center, 2005. URL: <http://ijcai.org/Proceedings/05/Papers/0372.pdf>.
- 11 Franz Baader, Ian Horrocks, Carsten Lutz, and Ulrike Sattler. *An Introduction to Description Logic*. Cambridge University Press, 2017. doi:<https://doi.org/10.1017/9781139025355>.
- 12 Franz Baader and Werner Nutt. Basic description logics. In *The Description Logic Handbook: Theory, Implementation, and Applications*, pages 43–95. Cambridge University Press, 2003. doi:<https://doi.org/10.1017/CB09780511711787>.
- 13 Chitta Baral. *Knowledge Representation, Reasoning and Declarative Problem Solving*. Cambridge University Press, 2010. doi:<https://doi.org/10.1017/CB09780511543357>.
- 14 Pietro Baroni, Francesca Toni, and Bart Verheij. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games: 25 years later. *Argument Comput.*, 11(1-2):1–14, 2020. doi:10.3233/AAC-200901.
- 15 Matti Berthold. On syntactic forgetting with strong persistence. In *Proceedings of the 19th International Conference on Principles of Knowledge Representation and Reasoning, KR 2022*, 2022. URL: <https://proceedings.kr.org/2022/5/>.
- 16 Matti Berthold, Ricardo Gonçalves, Matthias Knorr, and João Leite. A syntactic operator for forgetting that satisfies strong persistence. *Theory Pract. Log. Program.*, 19(5-6):1038–1055, 2019. doi:10.1017/S1471068419000346.
- 17 Matti Berthold, Matthias Knorr, and Daphne Odekerken. Forgettingweb. In *Proceedings of the 39th International Conference on Logic Programming*, volume 385, pages 321–323. EPTCS, 2023. doi:10.4204/EPTCS.385.
- 18 Philippe Besnard. *An introduction to default logic*. Symbolic computation. Springer, 1989. doi:<https://doi.org/10.1007/978-3-662-05689-9>.
- 19 Alexander Borgida, David Toman, and Grant E. Weddell. On referring expressions in query answering over first order knowledge bases. In *Proceedings of the 15th International Conference on Principles of Knowledge Representation and Reasoning, KR 2016*, pages 319–328. AAAI Press, 2016. URL: <http://www.aaai.org/ocs/index.php/KR/KR16/paper/view/12860>.
- 20 Elena Botoeva, Boris Konev, Carsten Lutz, Vladislav Ryzhikov, Frank Wolter, and Michael Zakharyashev. Inseparability and conservative extensions of description logic ontologies: A survey. In *Reasoning Web: Logical Foundation of Knowledge Graph Construction and Query Answering - 12th International Summer School 2016*, volume 9885 of *Lecture Notes in Computer Science*, pages 27–89. Springer, 2016. doi:10.1007/978-3-319-49493-7_2.
- 21 Ronald J. Brachman and Hector J. Levesque. *Knowledge Representation and Reasoning*. Elsevier, 2004. doi:<https://doi.org/10.1016/B978-1-55860-932-7.X5083-3>.
- 22 Gerhard Brewka, Thomas Eiter, and Miroslaw Truszczyński. Answer set programming at a glance. *Communications of the ACM*, 54(12):92–103, 2011. doi:10.1145/2043174.2043195.
- 23 Diego Calvanese, Giuseppe De Giacomo, and Maurizio Lenzerini. Reasoning in expressive description logics with fixpoints based on automata on infinite trees. In *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence, IJCAI 99*, pages 84–89. Morgan Kaufmann, 1999. URL: <http://ijcai.org/Proceedings/99-1/Papers/013.pdf>.
- 24 Ronnie Cann. *Formal Semantics: An Introduction*. Cambridge Textbooks in Linguistics. Cambridge University Press, 1993. doi:<https://doi.org/10.1017/CB09781139166317.002>.

42 Interpolation in Knowledge Representation

- 25 Valentin Cassano, Raul Fervari, Carlos Areces, and Pablo F. Castro. Interpolation and beth definability in default logics. In *Logics in Artificial Intelligence - 16th European Conference, JELIA 2019*, volume 11468 of *Lecture Notes in Computer Science*, pages 675–691. Springer, 2019. doi:10.1007/978-3-030-19570-0_44.
- 26 Giovanna D’Agostino and Marco Hollenberg. Logical questions concerning the mu-calculus: Interpolation, lyndon and los-tarski. *J. Symb. Log.*, 65(1):310–332, 2000. doi:10.2307/2586539.
- 27 Manuel de Sousa Ribeiro and João Leite. Aligning artificial neural networks and ontologies towards explainable AI. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021*, pages 4932–4940. AAAI Press, 2021. doi:10.1609/AAAI.V35I6.16626.
- 28 Warren Del-Pinto and Renate A. Schmidt. Abox abduction via forgetting in ALC. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019*, pages 2768–2775. AAAI Press, 2019. doi:10.1609/AAAI.V33I01.33012768.
- 29 Kevin Donnelly et al. SNOMED-CT: The advanced terminology and coding system for eHealth. *Studies in health technology and informatics*, 121:279, 2006. URL: <https://ebooks.iospress.nl/volumearticle/9130>.
- 30 Thomas Eiter and Gabriele Kern-Isberner. A brief survey on forgetting from a knowledge representation and reasoning perspective. *Künstliche Intell.*, 33(1):9–33, 2019. doi:10.1007/S13218-018-0564-6.
- 31 Patrice Enjalbert and Luis Fariñas del Cerro. Modal resolution in clausal form. *Theor. Comput. Sci.*, 65(1):1–33, 1989. doi:10.1016/0304-3975(89)90137-0.
- 32 Jorge Fandinno and Claudia Schulz. Answering the “why” in answer set programming — A survey of explanation approaches. *Theory Pract. Log. Program.*, 19(2):114–203, 2019. doi:10.1017/S1471068418000534.
- 33 Marie Fortin, Boris Konev, and Frank Wolter. Interpolants and Explicit Definitions in Extensions of the Description Logic EL. In *Proceedings of the 19th International Conference on Principles of Knowledge Representation and Reasoning, KR 2022*, pages 152–162, 8 2022. doi:10.24963/kr.2022/16.
- 34 Enrico Franconi and Volha Kerhet. Effective query answering with ontologies and DBoxes. In *Description Logic, Theory Combination, and All That - Essays Dedicated to Franz Baader on the Occasion of His 60th Birthday*, volume 11560 of *Lecture Notes in Computer Science*, pages 301–328. Springer, 2019. doi:10.1007/978-3-030-22102-7_14.
- 35 Enrico Franconi, Volha Kerhet, and Nhung Ngo. Exact query reformulation over databases with first-order and description logics ontologies. *J. Artif. Intell. Res.*, 48:885–922, 2013. doi:10.1613/JAIR.4058.
- 36 Maurice Funk, Jean Christoph Jung, Carsten Lutz, Hadrien Pulcini, and Frank Wolter. Learning description logic concepts: When can positive and negative examples be separated? In *Proceedings of the 28th International Joint Conference on Artificial Intelligence, IJCAI 2019*, pages 1682–1688, 2019. doi:10.24963/ijcai.2019/233.
- 37 Dov M. Gabbay and Larisa Maksimova. *Interpolation and Definability: Modal and Intuitionistic Logics*. Oxford University Press, 05 2005. doi:10.1093/acprof:oso/9780198511748.001.0001.

- 38 Dov M. Gabbay, David Pearce, and Agustín Valverde. Interpolable formulas in Equilibrium Logic and Answer Set Programming. *J. Artif. Intell. Res.*, 42:917–943, 2011. URL: <https://www.jair.org/index.php/jair/article/view/10743/25661>.
- 39 Dov M. Gabbay and Karl Schlechta. *A New Perspective on Nonmonotonic Logics*. Springer, 2016. doi:10.1007/978-3-319-46817-4.
- 40 Dov M. Gabbay, Renate A. Schmidt, and Andrzej Szalas. *Second-Order Quantifier Elimination - Foundations, Computational Aspects and Applications*, volume 12 of *Studies in logic : Mathematical logic and foundations*. College Publications, 2008. URL: <http://collegepublications.co.uk/logic/mlf/?00009>.
- 41 Fabien Gandon. *Distributed Artificial Intelligence And Knowledge Management: Ontologies And Multi-Agent Systems For A Corporate Semantic Web. (Intelligence Artificielle Distribuée Et Gestion Des Connaissances : Ontologies Et Systèmes Multi-Agents Pour Un Web Sémantique Organisationnel)*. PhD thesis, University of Nice Sophia Antipolis, France, 2002. URL: <https://tel.archives-ouvertes.fr/tel-00378201>.
- 42 Peter Gärdenfors. *Knowledge in flux: Modeling the dynamics of epistemic states*. The MIT press, 1988. doi:<https://doi.org/10.2307/2275379>.
- 43 Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. *Answer Set Solving in Practice*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2012. doi:10.2200/S00457ED1V01Y201211AIM019.
- 44 Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. Multi-shot ASP solving with clingo. *Theory and Practice of Logic Programming*, 19(1):27–82, 2019. doi:10.1017/S1471068418000054.
- 45 Michael Gelfond and Vladimir Lifschitz. The stable model semantics for logic programming. In *Logic Programming, Proceedings of the Fifth International Conference and Symposium*, pages 1070–1080. MIT Press, 1988.
- 46 Michael Gelfond and Vladimir Lifschitz. Classical negation in logic programs and disjunctive databases. *New Generation Computing*, 9(3-4):365–385, 1991. doi:<https://doi.org/10.1007/BF03037169>.
- 47 Silvio Ghilardi, Carsten Lutz, and Frank Wolter. Did I damage my ontology? A case for conservative extensions in description logics. In *Proceedings, Tenth International Conference on Principles of Knowledge Representation and Reasoning*, pages 187–197. AAAI Press, 2006. URL: <http://www.aaai.org/Library/KR/2006/kr06-021.php>.
- 48 Ricardo Gonçalves, Tomi Janhunen, Matthias Knorr, and João Leite. On syntactic forgetting under uniform equivalence. In *Logics in Artificial Intelligence - 17th European Conference, JELIA 2021, Proceedings*, volume 12678 of *Lecture Notes in Computer Science*, pages 297–312. Springer, 2021. doi:10.1007/978-3-030-75775-5_20.
- 49 Ricardo Gonçalves, Tomi Janhunen, Matthias Knorr, João Leite, and Stefan Woltran. Forgetting in Modular Answer Set Programming. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019*, pages 2843–2850. AAAI Press, 2019. doi:10.1609/AAAI.V33I01.33012843.
- 50 Ricardo Gonçalves, Matthias Knorr, and João Leite. You can’t always forget what you want: On the limits of forgetting in Answer Set Programming. In *ECAI 2016 - 22nd European Conference*

44 Interpolation in Knowledge Representation

- on *Artificial Intelligence - Including Prestigious Applications of Artificial Intelligence (PAIS 2016)*, volume 285 of *Frontiers in Artificial Intelligence and Applications*, pages 957–965. IOS Press, 2016. doi:10.3233/978-1-61499-672-9-957.
- 51 Ricardo Gonçalves, Matthias Knorr, and João Leite. Forgetting in answer set programming - A survey. *Theory Pract. Log. Program.*, 23(1):111–156, 2023. doi:10.1017/S1471068421000570.
- 52 Ricardo Gonçalves, Matthias Knorr, João Leite, and Stefan Woltran. On the limits of forgetting in Answer Set Programming. *Artif. Intell.*, 286:103–307, 2020. doi:10.1016/J.ARTINT.2020.103307.
- 53 Valentin Goranko and Martin Otto. Model theory of modal logic. In *Handbook of Modal Logic*, pages 249–329. Elsevier, 2007. doi:https://doi.org/10.1016/S1570-2464(07)80008-5.
- 54 Bernardo Cuenca Grau. Privacy in ontology-based information systems: A pending matter. *Semantic Web*, 1(1-2):137–141, 2010. doi:10.3233/SW-2010-0009.
- 55 Bernardo Cuenca Grau, Ian Horrocks, Yevgeny Kazakov, and Ulrike Sattler. Modular reuse of ontologies: Theory and practice. *J. Artif. Intell. Res.*, 31:273–318, 2008. doi:10.1613/JAIR.2375.
- 56 Andreas Herzig and Jérôme Mengin. Uniform interpolation by resolution in modal logic. In *Logics in Artificial Intelligence, 11th European Conference, JELIA 2008*, volume 5293 of *Lecture Notes in Computer Science*, pages 219–231. Springer, 2008. doi:10.1007/978-3-540-87803-2_19.
- 57 Jan Heuer and Christoph Wernhard. Synthesizing strongly equivalent logic programs: Beth definability for answer set programs via craig interpolation in first-order logic. In *Automated Reasoning - 12th International Joint Conference, IJCAR 2024*, volume 14739 of *Lecture Notes in Computer Science*, pages 172–193. Springer, 2024. doi:10.1007/978-3-031-63498-7_11.
- 58 Arend Heyting. Die formalen Regeln der intuitionistischen Logik. *Sitzungsberichte der Preussischen Akademie der Wissenschaften. Physikalisch-mathematische Klasse*, pages 42–56, 1930.
- 59 Eva Hoogland and Maarten Marx. Interpolation and definability in guarded fragments. *Studia Logica*, 70(3):373–409, 2002. doi:10.1023/A:1015154431342.
- 60 Eva Hoogland, Maarten Marx, and Martin Otto. Beth definability for the guarded fragment. In *Proceedings of the 6th International Conference on Logic Programming and Automated Reasoning, LPAR 1999*, pages 273–285. Springer, 1999. doi:10.1007/3-540-48242-3_17.
- 61 Ian Horrocks. Ontologies and the semantic web. *Commun. ACM*, 51(12):58–67, 2008. doi:10.1145/1409360.1409377.
- 62 Ian Horrocks, Peter F. Patel-Schneider, and Frank van Harmelen. From SHIQ and RDF to OWL: the making of a web ontology language. *J. Web Semant.*, 1(1):7–26, 2003. doi:10.1016/J.WEBSEM.2003.07.001.
- 63 Jean Christoph Jung, Jędrzej Kołodziejwski, and Frank Wolter. Computation of interpolants for description logic concepts in hard cases. In *Proceedings of the 38th International Workshop on Description Logics (DL 2025)*. CEUR-WS.org, 2025.
- 64 Jean Christoph Jung, Carsten Lutz, Mauricio Martel, and Thomas Schneider. Conservative extensions in horn description logics with inverse roles. *J. Artif. Intell. Res.*, 68:365–411, 2020. doi:10.1613/JAIR.1.12182.
- 65 Jean Christoph Jung, Carsten Lutz, Hadrien Pulcini, and Frank Wolter. Separating data examples by description logic concepts with restricted signatures. In *Proceedings of the 18th*

- International Conference on Principles of Knowledge Representation and Reasoning, KR 2021*, pages 390–399, 2021. doi:10.24963/kr.2021/37.
- 66 Jean Christoph Jung, Carsten Lutz, Hadrien Pulcini, and Frank Wolter. Logical separability of labeled data examples under ontologies. *Artif. Intell.*, 313:103785, 2022. doi:10.1016/j.artint.2022.103785.
 - 67 Jean Christoph Jung and Frank Wolter. Living without beth and craig: Definitions and interpolants in the guarded and two-variable fragments. In *Proceedings of the 36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021*, pages 1–14. IEEE, 2021. doi:10.1109/LICS52264.2021.9470585.
 - 68 Matthias Knorr and José Júlio Alferes. Preserving strong equivalence while forgetting. In *Logics in Artificial Intelligence - 14th European Conference, JELIA 2014. Proceedings*, volume 8761 of *Lecture Notes in Computer Science*, pages 412–425. Springer, 2014. doi:10.1007/978-3-319-11558-0_29.
 - 69 Daphne Koller and Nir Friedman. *Probabilistic Graphical Models — Principles and Techniques*. MIT Press, 2009. URL: <https://mitpress.mit.edu/9780262013192/probabilistic-graphical-models/>.
 - 70 Boris Konev, Carsten Lutz, Dirk Walther, and Frank Wolter. Formal properties of modularisation. In *Modular Ontologies: Concepts, Theories and Techniques for Knowledge Modularization*, volume 5445 of *Lecture Notes in Computer Science*, pages 25–66. Springer, 2009. doi:10.1007/978-3-642-01907-4_3.
 - 71 Boris Konev, Carsten Lutz, Dirk Walther, and Frank Wolter. Model-theoretic inseparability and modularity of description logic ontologies. *Artif. Intell.*, 203:66–103, 2013. doi:10.1016/J.ARTINT.2013.07.004.
 - 72 Boris Konev, Dirk Walther, and Frank Wolter. The logical difference problem for description logic terminologies. In *Automated Reasoning, 4th International Joint Conference, IJCAR 2008*, volume 5195 of *Lecture Notes in Computer Science*, pages 259–274. Springer, 2008. doi:10.1007/978-3-540-71070-7_21.
 - 73 Boris Konev, Dirk Walther, and Frank Wolter. Forgetting and uniform interpolation in large-scale description logic terminologies. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence, IJCAI 2009*, pages 830–835, 2009. URL: <http://ijcai.org/Proceedings/09/Papers/142.pdf>.
 - 74 Bogumil M Konopka. Biomedical ontologies—a review. *Biocybernetics and Biomedical Engineering*, 35(2):75–86, 2015. doi:<https://doi.org/10.1016/j.bbe.2014.06.002>.
 - 75 Patrick Koopmann. *Practical uniform interpolation for expressive description logics*. PhD thesis, University of Manchester, UK, 2015. URL: <https://ethos.bl.uk/OrderDetails.do?uin=uk.bl.ethos.674705>.
 - 76 Patrick Koopmann. LETHE: forgetting and uniform interpolation for expressive description logics. *Künstliche Intell.*, 34(3):381–387, 2020. doi:10.1007/S13218-020-00655-W.
 - 77 Patrick Koopmann and Jieying Chen. Deductive module extraction for expressive description logics. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, pages 1636–1643. ijcai.org, 2020. doi:10.24963/IJCAI.2020/227.
 - 78 Patrick Koopmann, Warren Del-Pinto, Sophie Tourret, and Renate A. Schmidt. Signature-based abduction for expressive description logics. In *Proceedings of the 17th International Conference*

- on *Principles of Knowledge Representation and Reasoning, KR 2020*, pages 592–602, 2020. doi:10.24963/KR.2020/59.
- 79 Patrick Koopmann and Renate A. Schmidt. Forgetting concept and role symbols in *ALCH*-ontologies. In *Proceedings of 19th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning, LPAR 2019*, volume 8312 of *Lecture Notes in Computer Science*, pages 552–567. Springer, 2013. doi:10.1007/978-3-642-45221-5\37.
 - 80 Emiel Krahmer and Kees van Deemter. Computational generation of referring expressions: A survey. *Comput. Linguistics*, 38(1):173–218, 2012. doi:10.1162/COLI\A\00088.
 - 81 Agi Kurucz, Frank Wolter, and Michael Zakharyashev. Definitions and (uniform) interpolants in first-order modal logic. In *Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning, KR 2023*, pages 417–428, 2023. doi:10.24963/KR.2023/41.
 - 82 Agi Kurucz, Frank Wolter, and Michael Zakharyashev. The interpolant existence problem for weak K4 and difference logic. In *Advances in Modal Logic, AiML 2024*, pages 465–484. College Publications, 2024.
 - 83 Jens Lehmann and Pascal Hitzler. Concept learning in description logics using refinement operators. *Machine Learning*, 78:203–250, 2010. doi:10.1007/S10994-009-5146-2.
 - 84 Domenico Lembo, Maurizio Lenzerini, Riccardo Rosati, Marco Ruzzi, and Domenico Fabio Savo. Inconsistency-tolerant semantics for description logics. In *Web Reasoning and Rule Systems - Fourth International Conference, RR 2010*, volume 6333 of *Lecture Notes in Computer Science*, pages 103–117. Springer, 2010. doi:10.1007/978-3-642-15918-3\9.
 - 85 Vladimir Lifschitz. What is answer set programming? In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 1594–1597. AAAI Press, 2008. URL: <http://www.aaai.org/Library/AAAI/2008/aaai08-270.php>.
 - 86 Vladimir Lifschitz. *Answer Set Programming*. Springer, 2019. doi:10.1007/978-3-030-24658-7.
 - 87 Fangzhen Lin and Raymond Reiter. Forget it! In *Proceedings of AAAI 1994*, 1994. URL: <https://cdn.aaai.org/Symposia/Fall/1994/FS-94-02/FS94-02-037.pdf>.
 - 88 Zhao Liu, Chang Lu, Ghadah Alghamdi, Renate A. Schmidt, and Yizheng Zhao. Tracking semantic evolutionary changes in large-scale ontological knowledge bases. In *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management*, pages 1130–1139. ACM, 2021. doi:10.1145/3459637.3482307.
 - 89 Michel Ludwig and Boris Konev. Practical uniform interpolation and forgetting for ALC tboxes with applications to logical difference. In *Proceedings of the 14th International Conference on Principles of Knowledge Representation and Reasoning, KR 2014*. AAAI Press, 2014. URL: <http://www.aaai.org/ocs/index.php/KR/KR14/paper/view/7985>.
 - 90 Michel Ludwig and Dirk Walther. Towards a practical decision procedure for uniform interpolants of el-tboxes - a proof-theoretic approach. In *Proceedings of 2nd Global Conference on Artificial Intelligence, GCAI 2016*, volume 41 of *EPiC Series in Computing*, pages 147–160. EasyChair, 2016. doi:10.29007/BSQM.
 - 91 Carsten Lutz. Complexity and succinctness of public announcement logic. In *5th International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS 2006)*, pages 137–143. ACM, 2006. doi:10.1145/1160633.1160657.

- 92 Carsten Lutz, Inanç Seylan, and Frank Wolter. An automata-theoretic approach to uniform interpolation and approximation in the description logic EL. In *Proceedings of the 13th International Conference on Principles of Knowledge Representation and Reasoning, KR 2012*. AAAI Press, 2012. URL: <http://www.aaai.org/ocs/index.php/KR/KR12/paper/view/4511>.
- 93 Carsten Lutz, Inanç Seylan, and Frank Wolter. The data complexity of ontology-mediated queries with closed predicates. *Logical Methods in Computer Science*, 15(3), 2019. doi:10.23638/LMCS-15(3:23)2019.
- 94 Carsten Lutz, Dirk Walther, and Frank Wolter. Conservative extensions in expressive description logics. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence, IJCAI 2007*, pages 453–458, 2007. URL: <http://ijcai.org/Proceedings/07/Papers/071.pdf>.
- 95 Carsten Lutz and Frank Wolter. Deciding inseparability and conservative extensions in the description logic $\mathcal{EL}$. *J. Symb. Comput.*, 45(2):194–228, 2010. doi:10.1016/J.JSC.2008.10.007.
- 96 Carsten Lutz and Frank Wolter. Foundations for uniform interpolation and forgetting in expressive description logics. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence, IJCAI 2011*, pages 989–995. IJCAI/AAAI, 2011. doi:10.5591/978-1-57735-516-8/IJCAI11-170.
- 97 Timothy S. Lyon and Jonas Karge. Constructive interpolation and concept-based both definability for description logics via sequents. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, pages 3484–3492. International Joint Conferences on Artificial Intelligence Organization, 8 2024. Main Track. doi:10.24963/ijcai.2024/386.
- 98 Larisa Maksimova. Intuitionistic logic and implicit definability. *Ann. Pure Appl. Log.*, 105(1-3):83–102, 2000. doi:10.1016/S0168-0072(99)00049-4.
- 99 John McCarthy. Circumscription — A form of non-monotonic reasoning. *Artif. Intell.*, 13(1-2):27–39, 1980. doi:10.1016/0004-3702(80)90011-9.
- 100 Deborah L. McGuinness and Alexander Borgida. Explaining subsumption in description logics. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence, IJCAI 95*, pages 816–821. Morgan Kaufmann, 1995. URL: <http://ijcai.org/Proceedings/95-1/Papers/105.pdf>.
- 101 Julián Méndez, Christian Alrabbaa, Patrick Koopmann, Ricardo Langner, Franz Baader, and Raimund Dachsel. Evonne: A visual tool for explaining reasoning with OWL ontologies and supporting interactive debugging. *Comput. Graph. Forum*, 42(6), 2023. doi:10.1111/CGF.14730.
- 102 Nadeschda Nikitina and Birte Glimm. Hitting the sweetspot: Economic rewriting of knowledge bases. In *Proceedings of the 11th International Semantic Web Conference, ISWC 2012*, volume 7649 of *Lecture Notes in Computer Science*, pages 394–409. Springer, 2012. doi:10.1007/978-3-642-35176-1_25.
- 103 Nadeschda Nikitina and Sebastian Rudolph. (Non-)succinctness of uniform interpolants of general terminologies in the description logic EL. *Artif. Intell.*, 215:120–140, 2014. doi:10.1016/J.ARTINT.2014.06.005.
- 104 Andreas Nonnengart, Hans Jürgen Ohlbach, and Andrzej Szalas. Elimination of predicate quantifiers. *Logic, Language and Reasoning: Essays in honour of Dov Gabbay*, pages 149–171, 1999. doi:https://doi.org/10.1007/978-94-011-4574-9_9.

48 Interpolation in Knowledge Representation

- 105 Hans Jürgen Ohlbach. SCAN - elimination of predicate quantifiers. In *Automated Deduction - CADE-13, 13th International Conference on Automated Deduction*, volume 1104 of *Lecture Notes in Computer Science*, pages 161–165. Springer, 1996. doi:10.1007/3-540-61511-3_77.
- 106 Ana Ozaki. Learning description logic ontologies: Five approaches. where do they stand? *Künstliche Intell.*, 34(3):317–327, 2020. doi:10.1007/S13218-020-00656-9.
- 107 David Pearce. Stable inference as intuitionistic validity. *Journal Log. Program.*, 38(1):79–91, 1999. doi:10.1016/S0743-1066(98)10015-8.
- 108 Abraham Robinson. A result on consistency and its application to the theory of definition. *Journal of Symbolic Logic*, 25(2):174–174, 1960. doi:10.2307/2964240.
- 109 Ana Armas Romero, Bernardo Cuenca Grau, and Ian Horrocks. More: Modular combination of OWL reasoners for ontology classification. In *Proceedings of the 11th International Semantic Web Conference, ISWC 2012*, volume 7649 of *Lecture Notes in Computer Science*, pages 1–16. Springer, 2012. doi:10.1007/978-3-642-35176-1_1.
- 110 Zeynep Saribatur and Thomas Eiter. Omission-based abstraction for answer set programs. *Theory Pract. Log. Program.*, 21(2):145–195, 2021. doi:10.1017/S1471068420000095.
- 111 Zeynep G. Saribatur, Matthias Knorr, Ricardo Gonçalves, and João Leite. On abstracting over the irrelevant in answer set programming. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning, KR 2024*, 2024. doi:10.24963/KR.2024/61.
- 112 Zeynep G. Saribatur and Stefan Woltran. Foundations for projecting away the irrelevant in ASP programs. In *Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning, KR 2023*, pages 614–624, 2023. doi:10.24963/KR.2023/60.
- 113 Zeynep Gozen Saribatur, Thomas Eiter, and Peter Schüller. Abstraction for non-ground answer set programs. *Artif. Intell.*, 300:103563, 2021. doi:10.24963/IJCAI.2022/807.
- 114 Karl Schlechta. *Formal Methods for Nonmonotonic and Related Logics — Vol II: Theory Revision, Inheritance, and Various Abstract Properties*. Springer, 2018. doi:10.1007/978-3-319-89650-2.
- 115 Stefan Schlobach. Explaining subsumption by optimal interpolation. In *Proceedings of 9th European Conference on Logics in Artificial Intelligence, JELIA 2004*, volume 3229 of *Lecture Notes in Computer Science*, pages 413–425. Springer, 2004. doi:10.1007/978-3-540-30227-8_35.
- 116 Inanç Seylan, Enrico Franconi, and Jos de Bruijn. Effective query rewriting with ontologies over dboxes. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence, IJCAI 2009*, pages 923–925, 2009. URL: <http://ijcai.org/Proceedings/09/Papers/157.pdf>.
- 117 Balder ten Cate. Interpolation for extended modal languages. *J. Symb. Log.*, 70(1):223–234, 2005. doi:10.2178/jsl/1107298517.
- 118 Balder ten Cate, Willem Conradie, Maarten Marx, and Yde Venema. Definitorially complete description logics. In *Proceedings of the 10th International Conference on Principles of Knowledge Representation and Reasoning, KR 2006*, pages 79–89. AAAI Press, 2006. URL: <http://www.aaai.org/Library/KR/2006/kr06-011.php>.
- 119 Balder ten Cate, Enrico Franconi, and Inanç Seylan. Beth definability in expressive description logics. *J. Artif. Intell. Res.*, 48:347–414, 2013. doi:10.1613/JAIR.4057.

- 120 Stephan Tobies. *Complexity results and practical algorithms for logics in knowledge representation*. PhD thesis, RWTH Aachen University, Germany, 2001. URL: <https://publications.rwth-aachen.de/record/52234>.
- 121 David Toman and Grant E. Weddell. First-order rewritability in ontology-mediated querying in horn description logics. In *Proceedings of Thirty-Sixth AAAI Conference on Artificial Intelligence, AAAI 2022*, pages 5897–5905. AAAI Press, 2022. doi:10.1609/AAAI.V36I5.20534.
- 122 Ly Ly T. Trieu, Tran Cao Son, and Marcello Balduccini. xASP: An explanation generation system for answer set programming. In *Proceedings of the 16th Conference on Logic Programming and Nonmonotonic Reasoning, LPNMR 2022*, volume 13416 of *Lecture Notes in Computer Science*, pages 363–369. Springer, 2022. doi:10.1007/978-3-031-15707-3_28.
- 123 Frank van Harmelen, Vladimir Lifschitz, and Bruce Porter. *Handbook of Knowledge Representation*. Elsevier Science, San Diego, CA, USA, 2007.
- 124 Yisong Wang, Kewen Wang, and Mingyi Zhang. Forgetting for answer set programs revisited. In *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence*, pages 1162–1168. IJCAI/AAAI, 2013. URL: <https://www.ijcai.org/Proceedings/13/Papers/175.pdf>.
- 125 Yisong Wang, Yan Zhang, Yi Zhou, and Mingyi Zhang. Forgetting in logic programs under strong equivalence. In *Principles of Knowledge Representation and Reasoning: Proceedings of the Thirteenth International Conference, KR 2012*. AAAI Press, 2012. URL: <http://www.aaai.org/ocs/index.php/KR/KR12/paper/view/4389>.
- 126 Yisong Wang, Yan Zhang, Yi Zhou, and Mingyi Zhang. Knowledge forgetting in Answer Set Programming. *J. Artif. Intell. Res.*, 50:31–70, 2014. doi:10.1613/JAIR.4297.
- 127 Zhe Wang, Kewen Wang, Rodney W. Topor, and Jeff Z. Pan. Forgetting for knowledge bases in DL-Lite. *Ann. Math. Artif. Intell.*, 58(1-2):117–151, 2010. doi:10.1007/S10472-010-9187-9.
- 128 Zhe Wang, Kewen Wang, Rodney W. Topor, and Xiaowang Zhang. Tableau-based forgetting in *ALC* ontologies. In *Proceedings of 19th European Conference on Artificial Intelligence, ECAI 2010*, volume 215 of *Frontiers in Artificial Intelligence and Applications*, pages 47–52. IOS Press, 2010. URL: <https://ebooks.iospress.nl/volumearticle/5737>.
- 129 Hui Yang, Patrick Koopmann, Yue Ma, and Nicole Bidoit. Efficient computation of general modules for *ALC* ontologies. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023*, pages 3356–3364. ijcai.org, 2023. doi:10.24963/IJCAI.2023/374.
- 130 Hui Yang, Yue Ma, and Nicole Bidoit. Efficient extraction of el-ontology deductive modules. In *Proceedings of 37th AAAI Conference on Artificial Intelligence, AAAI 2023*, pages 6575–6582. AAAI Press, 2023. doi:10.1609/AAAI.V37I5.25808.
- 131 Yizheng Zhao and Renate A. Schmidt. FAME(Q): an automated tool for forgetting in description logics with qualified number restrictions. In *Proceedings of the 27th International Conference on Automated Deduction, CADE 2019*, volume 11716 of *Lecture Notes in Computer Science*, pages 568–579. Springer, 2019. doi:10.1007/978-3-030-29436-6_34.